

MINISTRY OF EDUCATION AND RESEARCH
Technical University of Moldova
Faculty of Computers, Informatics, and Microelectronics
Department of Software Engineering and Automation

**SISTEM AUTOMATIZAT DE CREARE A
OPERATORILOR VOCALI INTELIGENȚI PENTRU
COMPANII. DEZVOLTAREA INTERFEȚEI
GRAFICE.**

**AUTOMATED SYSTEM FOR CREATING
INTELLIGENT VOICE OPERATORS FOR
COMPANIES. FRONT-END DEVELOPMENT.**
Undergraduate Internship Report

Student: _____ **Ostafi Eugen, gr. FAF-222**

Coordinator: _____ **Gavriliță Mihail, univ. assist.**

Chisinau, 2026

TABLE OF CONTENTS

INTRODUCTION	2
1 DOMAIN ANALYSIS	5
1.1 Problem Definition	5
1.2 Domain Description	6
1.3 Existing Solutions and Comparative Analysis	11
1.4 Target Audience	13
1.5 SWOT Analysis	15
2 Functional Requirements	20
2.1 Functional Requirements Specification	20
2.2 Non-functional Requirements Specification	24
3 System Design	27
3.1 System Design	27
3.2 Use Case Diagrams	28
3.3 Sequence Diagrams	33
3.4 Activity Diagrams Collaboration Diagram	36
3.5 State Diagrams Class Diagrams	38
4 System Implementation	41
4.1 Code-Level Description by Modules	41
4.2 System Testing	47
5 Product Documentation	53
5.1 Landing Page	53
5.2 Registration Page	54
5.3 Login Page	56
5.4 Dashboard Overview	57
5.5 Working Hours	58
5.6 Services	59
5.7 FAQ / Knowledge Base	61
5.8 Call Logs	62
5.9 Assistant Settings	63
CONCLUSIONS	65
BIBLIOGRAPHY	66

INTRODUCTION

Small and medium enterprises are really important, for our economy. They help a lot of people get jobs. They come up with new ideas. Small and medium enterprises also help the area where they are based.

Small and medium enterprises have a lot of problems though. They have trouble running things and doing business because of the way they are set up. This makes it hard for small and medium enterprises to compete with companies.

One of the problems that small and medium enterprises face is talking to their customers. Small and medium enterprises have a time managing what they say to their customers especially when they talk to them on the phone. When you are doing business these days things are very competitive. They happen really fast. Customers want to get information away they want to deal with professionals and they want service they can count on. This is what customers expect no matter how big or small the business is that they are contacting. The customers just want an experience, with the business so they want the business to be reliable and helpful.

People expect a lot from businesses these days. This is because technology has improved much and everyone is using computers and phones. Big companies have started using good systems to help their customers like call centers and computers that can talk to people. These systems are always available. Can help people right away. Now people want small businesses to be just as good, at helping them. The problem is that small businesses do not have a lot of money or people to work on these systems. Small businesses also do not have the technology they need to make these systems work. There is a difference between what customers want and what small and medium enterprises can do. This difference is an issue that affects how happy customers are with a company, what they think of the brand and whether the business can keep running. The customer expectations and the abilities of medium enterprises are not matching up and this is causing problems for customer satisfaction, brand perception and business sustainability. The gap, between customer expectations and small and medium enterprises capabilities is getting bigger. It is a critical problem.

Phone calls are still an important way for small and medium sized businesses to talk to people. This is especially true for places like shops, restaurants and services that people use every day. People often call these businesses to ask questions like what time they are open if they have something, in stock how much things cost or where they are located. Usually the people who own the business or work there have to answer these calls themselves. They are already doing a lot of jobs so it can be hard for them to answer the phone all the time. When it is busy the staff might not be able to pick up the phone, which means they miss out on chances to help people and the customers get annoyed. People who study how customers act have found that

a lot of customers will not try to call if their call is not answered. They will just choose another company that seems easier to get in touch with. This means that the first company loses the customer because they did not answer the phone. Customer behavior is really important. Companies should pay attention to it. If a customer calls and nobody answers they are likely to call someone, like a competitor of the company. Customer behavior research shows this is what happens when customers do not get an answer.

The problem gets even worse when it is outside business hours. Medium sized businesses or SMEs do not usually have customer support available all the time like big companies do. This is because SMEs have to be careful with how money they spend. So when someone calls SMEs in the evening or at night or on the weekends nobody answers. A lot of the time people who call SMEs just have questions that a machine could answer.. Smes do not have the right tools to make this happen. This means SMEs have to deal with not being able to help their customers all the time. Over time this makes customers trust SMEs less and less. It also makes customers less loyal to SMEs. This is bad for SMEs because it makes it harder for them to compete with businesses, in their area. SMEs have to think about customer trust and loyalty when they are trying to sell things to people in their markets.

The thing about intelligence is that it is getting better and better. This is really true when we talk about intelligence and things like speech recognition and natural language processing and text-, to-speech synthesis. These things are making it possible for companies to use machines to talk to customers.

Artificial intelligence voice assistants can now understand what people are saying. Artificial intelligence can figure out what people want. Then give them answers that sound like a real person is talking. This is happening now.

These artificial intelligence technologies can really change the way companies do customer service. Artificial intelligence can give people answers away and always give the same answer. Artificial intelligence can also handle a lot of people at the time.

Most of the time artificial intelligence customer service is used by big companies and call centers. These systems usually need a lot of setup and special knowledge to work properly. They cost a lot of money. This makes them not a good choice, for Medium Enterprises. Small and Medium Enterprises often find that these systems are too much to handle.

This project is about making an Automated Voice Assistant System for Businesses. The Automated Voice Assistant System is designed for Small Businesses. It will help them with things they need. It will answer phone calls from customers give them the information and be available all the time. The Automated Voice Assistant System will do tasks that staff have to do over and over so staff will have work to do. This means customers will be happier, with the service they get from Businesses and their Automated Voice Assistant System.

The system they are talking about is based on a main ideas: it is simple, affordable, reliable and it

can grow with the business. The system is simple so that people who own a business and have computer skills can set it up and manage it without needing help from a technical person. It is affordable because it is made for businesses that do not have a lot of money to spend and it is cheaper than hiring more staff. The system is reliable which means it always gives the answers and it can grow with the business so it can do more things in the future like schedule appointments take orders or speak different languages. The system is reliable. That is important for business owners. The system can. That means it can do more things like scheduling appointments for the business taking orders, for the business or supporting multiple languages for the business.

This project is not just about meeting its goals. It also thinks about how it affects the economy and society. The project helps medium sized enterprises or SMEs use artificial intelligence to automate tasks. This helps these small businesses keep up with the times and stay competitive.

The system is good for people who work at SMEs too. It helps them use their time better. Of doing boring administrative work employees can focus on tasks that need human thought and personal touch. This is really important for SMEs because they need to use their people in the way possible. The project is, about helping SMEs and their employees by using artificial intelligence in a smart way.

This project is really important. That is shown by the research that has been done and the early studies that have been carried out. When people who own medium sized businesses were asked some questions it was found that they miss a lot of calls from customers they spend a lot of time answering the same questions over and over again and they are very interested, in finding ways to automate their businesses that do not cost too much money. Most of the people who were asked said they would be happy to use an automated voice assistant if it was easy to set up and did not cost much. These results show that the problem that this project is trying to solve is a problem that many people face. The project is trying to solve this problem. That is why it is so important. The problem of missed customer calls and repetitive inquiries is an one and this project is trying to find a way to make it easier for small and medium sized businesses to deal with these issues.

In conclusion, the Automated Voice Assistant System for Small Businesses is motivated by a clear and pressing need within the SME sector. It seeks to bridge the gap between rising customer expectations and limited business resources by leveraging modern AI technologies in a practical and accessible manner. This introduction establishes the context and justification for the project, laying the foundation for the subsequent chapters, which will explore the domain analysis, system requirements, architectural design, and implementation approach in detail.

1 DOMAIN ANALYSIS

1.1 Problem Definition

Small and medium enterprises (SMEs) rely heavily on fast and clear communication with customers. In many local businesses—such as retail shops, food services, salons, repair services, or small clinics—the phone call is still one of the most important channels for getting information, placing requests, or scheduling appointments. At the same time, customers also expect businesses to respond on messaging platforms and to provide quick answers even outside working hours. The main difficulty is that SMEs usually do not have a dedicated receptionist or customer service department. In reality, the same small team that delivers the service (or sells the products) must also answer calls and messages, often during peak hours when they are already busy.

The core problem addressed by this project is that SMEs often cannot provide consistent, immediate, and professional customer interaction across voice and messaging channels, mainly because of limited staff, limited time, and limited budgets. Missed calls, late replies, and short or unclear explanations are not necessarily caused by “poor service,” but by operational constraints. However, from the customer’s point of view, slow communication often feels like lack of interest or lack of professionalism. This gap can directly lead to lost clients, because customers frequently choose the option that responds first—especially when the request is simple and urgent (for example, “Are you open?”, “Do you have availability today?”, or “How much does this cost?”).

Another important part of the problem is that a large share of customer interactions are repetitive and predictable. Many customers ask the same questions: working hours, address, services, prices, booking, delivery, payment methods, return policies, and so on. For SMEs, repeatedly answering these questions consumes attention that could be used for serving customers, improving the business process, or generating revenue. In other words, employees spend valuable time on tasks that can be standardized, automated, and delivered consistently.

Customer expectations have also changed in recent years. People are used to getting information instantly through apps and online services. Even if they understand that a small business is not a corporation, they still compare their experience to “always available” services. As a result, SMEs face pressure to provide *near real-time responses* and *availability outside normal working hours*, but they cannot realistically keep staff dedicated to communication 24/7. This makes the problem not only about answering calls, but about competitiveness in a market where communication speed and clarity strongly influence customer choices..

From a technical perspective, the difficulty is not simply “building a chatbot.” The real-world domain

problem includes several constraints that must be satisfied at the same time:

- voice channel support (real phone calls, not only website chat);
- low latency (the assistant must respond quickly enough for natural conversation);
- accurate intent understanding (correctly interpreting what the customer wants);
- up-to-date business information (hours, availability, services, pricing—stored in a database and easy to update);
- affordable deployment and maintenance (aligned with SME budgets);
- ease of setup (SMEs often lack technical staff);
- privacy and data protection (customer calls/messages may contain personal data, so storage and processing must be treated responsibly, especially under GDPR-type principles in Europe)[1];

Because of these factors, the domain problem is best viewed as an operational efficiency and service accessibility problem, not just an “AI feature.” SMEs need a reliable first point of contact that can handle routine requests automatically, reduce missed opportunities, and escalate complex situations to humans when needed.

For this reason, the project proposes an Automated Voice Assistant System for Small Businesses that automates routine customer interactions using speech-to-text, natural language processing, text-to-speech, and real-time database access. The goal is to reduce response times, improve consistency and professionalism across voice channels. In practical terms, the system acts like a virtual receptionist that is available 24/7, helping SMEs meet modern communication expectations without hiring additional full-time staff.

1.2 Domain Description

The domain of this project is *automated customer communication for small and medium enterprises (SMEs)*, with a focus on *voice-based interaction through phone calls*. In many SMEs, the phone remains a critical channel for customer interaction because it is fast, familiar, and does not require customers to install new applications or create accounts. At the same time, SMEs are typically limited in staff and time, which makes it difficult to respond consistently during busy hours or outside working hours. As a result, this domain sits at the intersection of *customer service operations*, *telephony systems*, and *conversational AI*.

Unlike large companies that can organize call centers, deploy CRM systems, and assign dedicated employees to handle inquiries, SMEs often work with informal, human-dependent communication workflows. A single missed call can mean a missed appointment or a lost order, especially in service businesses where customers contact multiple providers and choose whoever answers first. Therefore, the domain is not only about automation for convenience; it is directly connected to business efficiency, competitiveness, and

customer retention.

SMEs are the dominant type of organization in most economies. In the European Union, SMEs represent *99% of all businesses* [2]. This is relevant because improvements in SME operations scale to broad economic impact. SMEs usually face a specific set of constraints:

- *limited staff and multitasking roles*: employees often combine sales/service delivery with administrative tasks like answering calls;
- *no dedicated customer service processes*: there is rarely a formal script or standardized handling procedure;
- *budget constraints*: hiring additional staff or adopting enterprise software is often too expensive;
- *peak-hour overload*: calls increase during certain times of day or seasonal periods, exactly when staff are busiest.

Because of these constraints, customer communication in SMEs becomes irregular: calls may go unanswered, messages may be delayed, and answers may vary depending on which employee responds. In domains such as restaurants, clinics, beauty salons, repair services, and retail shops, these issues are amplified because calls often concern immediate decisions (availability, opening hours, booking) rather than long-term planning.

1.2.1 Local Context and Practical Constraints of Voice-Based SME Systems

Since the proposed solution is intended for implementation in the Republic of Moldova, it is important to describe the domain not only in general terms, but also in the local context. Moldova's economy is strongly influenced by small businesses, micro-enterprises, and self-employment activities, which generally implies the same constraints described above: limited personnel, multitasking roles, and limited budgets for customer service and digital transformation.

From a statistical perspective, the National Bureau of Statistics of the Republic of Moldova (NBS) publishes official information about the structure of enterprises by size. These statistics show that the enterprise ecosystem is dominated by *small and medium enterprises*, while large enterprises represent only a small fraction of the total [3]. This reinforces the idea that solutions designed specifically for SMEs are more relevant for Moldova than enterprise-only platforms.

In addition, Moldova has been actively advancing its digital transformation agenda in both public and private sectors, which supports the feasibility of adopting automation tools. National strategies and policy documents emphasize digitalization and the development of the information society, which indirectly encourages SMEs to adopt modern tools for communication and customer service [4]. For the proposed project, this context is important because an automated assistant requires basic digital readiness: stable internet connectivity, willingness to use web dashboards, and openness to cloud/self-hosted systems.

At the same time, local realities also influence requirements. Many businesses in Moldova operate with:

- *high reliance on phone calls* for confirmations and appointments;
- *informal communication habits* (quick calls rather than structured forms);
- *strong customer preference for immediate answers*.

These conditions match the target domain well: if a business misses calls during peak hours, it may lose customers to competitors, particularly in sectors with high competition in urban areas (e.g., Chişinău).

Finally, Moldova’s regulatory environment should be considered. While the system is not a legal product, it may process personal data (phone numbers, names, appointment details). Moldova has a national legal framework for personal data protection, and the National Center for Personal Data Protection provides guidance and enforcement. Including this is relevant because SMEs will be more willing to adopt the solution if data handling is clear and responsible [5].

A defining property of this domain is that a large portion of interactions are *high-frequency and low-complexity*. Many callers ask similar questions. Although details vary by industry, common intent categories include:

- *business information requests*: working hours, address, directions, contact details;
- *service or product questions*: price, duration, what is included, availability of a product or service;
- *appointments and reservations*: booking, rescheduling, canceling, confirmation of an existing appointment;
- *orders and delivery*: delivery time, pickup instructions, order confirmation, basic status requests;
- *escalation*: request to speak with a human (manager/owner) or leave a message.

From a domain perspective, these intents are important because they define both the *knowledge scope* (what the assistant must know) and the *dialogue patterns* (how the assistant must ask questions, confirm details, and end calls politely). The objective is not to replace every possible human conversation, but to automate the most repetitive and operationally expensive interactions, while supporting escalation for cases that exceed the assistant’s confidence or authority.

Why voice automation is more challenging than text chat ? It is common to compare voice assistants to chatbots, but in practice, voice interaction introduces additional constraints that define the domain requirements. In a voice call, the user expects a natural rhythm. Delays and misunderstandings feel more frustrating than in text, because the caller cannot easily “scan” previous messages or silently correct their input. As a result, an SME voice assistant must address several challenges:

Phone audio quality is typically limited, and many calls happen in noisy environments (shops, streets, vehicles). Accents, fast speech, and background noise can significantly reduce speech recognition accuracy. Modern deep learning-based automatic speech recognition (ASR) systems such as Whisper have

been shown to provide robust performance across diverse conditions and are therefore suitable candidates for real-world deployment [6].

In live calls, response time is part of perceived quality. Even if the answer is correct, excessive delays can cause the interaction to feel broken. This domain requirement affects architectural decisions: models, infrastructure, and integrations must be chosen to minimize end-to-end latency.

A voice assistant must detect when the caller finished speaking and when it is appropriate to respond. It must also handle interruptions: callers might start talking before the assistant finishes. Telephony-oriented dialogue design often requires short prompts, confirmations, and fallback strategies.

Misrecognition and ambiguity are unavoidable in real calls. The domain therefore requires careful handling of uncertainty (e.g., asking a clarifying question or repeating a summary), because repeated mistakes reduce trust quickly.

These constraints explain why an automated voice assistant for SMEs is not simply a chatbot with speech added. The system must be designed for the realities of telephony and human conversation patterns.

1.2.2 Domain Analysis and Requirements for Automated SME Communication Systems

The assistant must operate inside the telephony ecosystem. This includes receiving calls, playing prompts, collecting speech input, and generating spoken responses. In practice, telephony integration is often implemented using a provider such as Twilio, which exposes programmable voice APIs and call control mechanisms [7]. For SMEs, this approach has two advantages:

- it reduces setup complexity (no need to manage telecom infrastructure);
- it supports rapid prototyping and piloting with minimal upfront cost.

At the same time, the domain also includes cost and control considerations. As the number of calls grows, per-minute fees may become significant, which motivates considering self-hosted or lower-cost telephony solutions (e.g., Asterisk) depending on the business scale. Thus, the domain includes a trade-off between ease of adoption and long-term operational cost.

A critical domain-specific requirement is that the assistant must provide *correct and up-to-date information*. Many assistant failures in real deployments happen not because the AI cannot speak, but because the information provided is outdated or inconsistent with reality. Many SMEs update schedules, prices, and availability frequently. For this reason, the domain strongly benefits from integrating a *database-backed knowledge layer*. A relational database such as PostgreSQL is a practical choice because it supports structured entities and transactional updates [8]. Typical data entities relevant to the SME assistant domain include:

- business profile (name, address, contacts);
- opening hours and exceptions;

- service or product catalog and pricing;
- appointment slots and bookings (for service businesses);
- call logs and interaction history (for analytics and debugging).

In addition, the domain requires that business owners or staff can update this information without programming. This motivates the use of a simple management interface (dashboard) as part of the broader system context.

The domain is relevant not only because SMEs experience communication overload, but also because SMEs represent the majority of enterprises. This is true at EU level [2] and also in Moldova, where official statistics indicate that the economy is dominated by enterprises in the SME category [3].

Additionally, customer expectations are shaped by modern digital communication. Messaging platforms are extremely widespread; WhatsApp reports *over 2 billion users* globally [9]. Even if this project focuses on voice calls, this statistic supports the argument that customers increasingly expect quick responses and efficient interaction. In practice, these expectations transfer to phone calls as well: users still want immediate answers rather than waiting for manual callbacks.

Customer calls and associated metadata (phone numbers, names, appointment times, and conversation content) can qualify as personal data. If the system records calls or stores transcripts, privacy risks increase. In the European context, data processing must follow GDPR principles [1]. For Moldova, personal data processing is regulated by national legislation, and the domain analysis must acknowledge that SMEs may need to comply with local data protection requirements as well [5].

Even when the project is academic, considering these constraints is important because they influence design decisions:

- whether to store audio recordings or only transcripts;
- retention periods for logs;
- access control to call data.
- transparency about third-party services used for ASR/TTS.

In summary, the domain addressed by the project can be characterized as follows:

- *high volume of repetitive interactions*, typically short and operational;
- *strong time sensitivity*: missed calls and slow answers can lead to direct business loss;
- *voice-specific technical constraints*: ASR accuracy, latency, turn-taking, error recovery;
- *need for structured, maintainable business knowledge*: database-backed answers and easy updates;
- *operational and regulatory constraints*: affordability, simplicity, and privacy compliance (EU and Moldova).

These characteristics justify the development of an Automated Voice Assistant System tailored to SMEs: a system that focuses on reliability, speed, and practical task completion in real phone call scenarios,

rather than on purely experimental conversational capabilities.

1.3 Existing Solutions and Comparative Analysis

In order to position the proposed Automated Voice Assistant System within the current market, we analyzed several well-known platforms that offer overlapping functionality. The goal of this comparison is not to claim that these competitors are “bad” solutions. In fact, they are mature and widely adopted. However, they are usually designed for *large-scale* customer support environments and often assume that the client has technical staff and a larger budget. For small businesses in Moldova, the main question is whether these platforms can be adopted quickly, affordably, and with minimal technical complexity.

For this comparative analysis, three relevant solutions were selected:

- *google Dialogflow*: a conversational AI platform used for building chatbots and voice agents [10];
- *amazon Connect*: a cloud contact center platform that can be extended with AI services for speech and chat automation [11];
- *twilio Flex and Twilio Autopilot*: Twilio Flex is a programmable contact center platform, while Twilio Autopilot was Twilio’s assistant-building product (now discontinued) [12, 13].

Table 1.1 — Comparative analysis of existing solutions

Competitor	Strengths	Weaknesses
Google Dialogflow	– Strong intent recognition and NLP capabilities. – Supports voice and chat agent design, with integrations in the Google ecosystem. – Includes tooling for conversation flows, contexts, and fulfillment via webhooks.	– Setup and maintenance usually require technical expertise. – Costs can grow with usage and additional services. – Feature richness can be excessive for SMEs.
Amazon Connect	– Enterprise-grade reliability and scalability. – Pay-as-you-go pricing. – Integration with AWS services.	– High initial setup complexity. – Difficult pricing estimation. – Oriented toward large contact centers.
Twilio Flex + Autopilot	– Highly customizable. – Strong developer ecosystem.	– Autopilot discontinued. – Requires development effort. – Not plug-and-play for SMEs.

Google Dialogflow is one of the most popular platforms for building conversational agents. It provides tools for defining intents, training phrases, entities, contexts, and integration logic through webhooks [10]. In theory, Dialogflow can support SME scenarios well because many routine customer interactions can be modeled as intents (e.g., opening hours, prices, booking requests). Dialogflow also benefits from Google’s broader cloud ecosystem, which can simplify integration with other services if a business already uses Google Cloud.

Strengths. The main advantage of Dialogflow is its maturity and NLP capabilities. It is suitable for intent classification and structured conversations, and it supports multiple channels (chat interfaces and voice integrations depending on configuration) [10]. For organizations with technical staff, it can be a powerful tool for creating stable conversational experiences.

Weaknesses. In the SME context, Dialogflow can become difficult because it often requires technical knowledge to design, train, and maintain the agent properly (intent coverage, fallback intents, webhook integration, deployment). In practice, many small businesses do not have developers, and paying an external specialist may be too costly relative to the value of automating a few repetitive call types. Additionally, the platform includes many advanced features that are not always necessary for small businesses seeking a simple virtual receptionist.

Amazon Connect is a cloud-based contact center solution that provides call routing, IVR flows, queue management, and integrations with AWS services [11]. The platform targets organizations that need reliability, scaling, and advanced contact center features. AI-driven automation can be achieved through integration with other AWS services (for example speech processing and conversational services), which makes it flexible for building advanced workflows.

Strengths. Amazon Connect is highly reliable and scalable, and the pay-as-you-go model can be attractive for organizations that grow over time [11]. It is well-suited for environments where many agents handle calls and where advanced analytics or monitoring is required.

Weaknesses. For SMEs, Amazon Connect is often too heavy. Even if the pricing model is usage-based, the overall solution requires AWS knowledge, configuration effort, and often multiple connected services. In real adoption scenarios, this increases the barrier to entry for micro-businesses or small companies in Moldova that want a practical solution without building a contact center infrastructure. The product is also designed around the idea of agent queues and contact center operations, which may not match the simpler SME workflow.

Twilio is well known for programmable communication APIs. Twilio Flex is a programmable contact center platform that enables developers to build customized customer service interfaces and integrate multiple channels [12]. In the past, Twilio Autopilot provided a way to build conversational assistants, but it has been discontinued [13]. This is an important point from a risk perspective: a discontinued product introduces uncertainty and maintenance problems for long-term deployments.

Strengths. Twilio Flex offers strong integration capabilities for voice and messaging, and the Twilio ecosystem is developer-friendly with clear documentation and APIs [12]. For teams with software development capacity, Twilio can support complex workflows and custom business logic.

Weaknesses. In the context of SMEs (especially micro-businesses), Twilio Flex is not a “ready-made assistant.” It is a flexible platform that requires development effort. Moreover, the discontinuation

of Autopilot highlights a stability risk when relying on a specific vendor product for conversational logic [13]. For this reason, Twilio-based architectures often work best when the conversational intelligence is implemented independently (e.g., via self-hosted NLP or external models) while Twilio is used primarily as the telephony layer.

Based on this analysis, the major limitation of existing market solutions is not their technical capability, but their *mismatch with SME constraints*. Enterprise platforms are typically designed for organizations with technical teams, formal contact center processes, and larger budgets. In Moldova, where the economy is strongly dominated by SMEs [3], adoption barriers such as setup complexity, unclear total cost, and the need for specialized staff become critical.

Therefore, the proposed Automated Voice Assistant System is more appropriate in this specific context because it is designed from the start for:

- *fast deployment and simple configuration* (SME-friendly);
- *cost control* through modular architecture and the option to self-host key components;
- *voice-first interaction* as a realistic channel for local SMEs;
- *database-driven answers* to ensure correct and up-to-date business information;
- *vendor independence* for conversational logic (reducing risk from discontinued services).

In short, while Dialogflow, Amazon Connect, and Twilio Flex are strong platforms, the proposed solution is better aligned with the practical needs of Moldovan SMEs: minimal operational overhead, predictable adoption effort, and a focus on the most common real-world phone-based customer interactions.

1.4 Target Audience

This section identifies the target audience of the Automated Voice Assistant System and explains why these user groups are relevant for the proposed solution. Understanding who will use the system (directly and indirectly) is important because it influences both functional requirements (what the assistant must do) and non-functional requirements (ease of use, reliability, cost, privacy).

The primary audience for the system consists of *owners and managers of small and medium enterprises (SMEs) in the Republic of Moldova*. This group is the most relevant because they are the main decision-makers: they decide whether to adopt the system, pay for it, and integrate it into daily business operations. The solution is designed specifically for businesses that:

- have a small staff (often fewer than 20 employees);
- handle customer communication directly through phone calls;
- do not have a dedicated receptionist or customer support role;
- are sensitive to operational costs and require clear, predictable value from new tools.

This target group is significant in Moldova because the enterprise ecosystem is largely dominated

by SMEs [3]. As a result, an automation tool aimed at SMEs is more likely to match the reality of the local economy than enterprise-only contact center solutions.

While the assistant can be adapted to many sectors, the most suitable SME categories in Moldova include:

- *retail stores and small local shops* (availability, prices, working hours, basic product questions);
- *food services* (restaurants, cafes, delivery kitchens) (menu questions, hours, reservations, delivery/pickup information);
- *local service providers* (beauty salons, repair services, tutoring centers, fitness studios) (appointments, pricing, service details);
- *small private clinics and cabinets* where calls relate to schedules and basic information (within privacy limits).

These businesses are typically characterized by frequent customer calls with repetitive requests, which makes them strong candidates for automation.

From a practical point of view, the system must be usable by people who are not highly technical. In Moldova, many SME owners and managers have *moderate digital literacy*: they use smartphones, messaging apps, social media pages, and online banking, but they do not usually have experience with configuring cloud platforms, telephony routing rules, or machine learning systems.

Therefore, the system must support:

- *simple setup and configuration* (clear onboarding, minimal technical steps);
- *easy updates to business data* (working hours, services, prices) through a dashboard;
- *transparent operation and control* (ability to review call logs, understand what the assistant answers, and override information when needed).

1.4.1 Audience Analysis and Functional Needs

A second important audience consists of *staff members working in SMEs*. Even if staff do not make purchasing decisions, they are direct beneficiaries and indirect users of the system. In small businesses, staff are often interrupted by calls while serving customers. This affects productivity and can reduce service quality for customers who are physically present.

By handling routine calls and frequently asked questions, an automated assistant can reduce interruptions and allow staff to focus on their main tasks. In practice, staff members need:

- fewer disruptions during peak hours;
- clear escalation logic (when the assistant forwards calls or notifies humans);
- trust that the assistant provides correct information.

Their feedback is also valuable for system tuning, because staff usually know which questions are

repeated most often and which answers must always remain consistent.

The final (and most important) users in terms of interaction are the *customers* who call the business. They do not install anything and typically do not know what platform is used; they only experience the service quality during the call. For customers, the main expectations are:

- *fast response* (reduced waiting time);
- *clear and correct answers*;
- *natural interaction* (short prompts, no unnecessary complexity);
- *availability outside working hours* for basic information.

In addition, customers often compare their experience across businesses. If one business answers instantly (even through automation) while another does not, the first business can gain a competitive advantage. This is especially relevant in urban markets such as Chişinău, where many service providers offer similar products and customers can switch easily.

In conclusion, the Automated Voice Assistant System targets Moldovan SMEs where customer communication is frequent, repetitive, and time-sensitive. The primary users are business owners and managers who need a cost-effective solution that does not require advanced technical knowledge. Secondary users include staff members who benefit from fewer interruptions, while end users are customers who receive faster and more consistent communication. Because the Moldovan economy is dominated by SMEs [3], and because local businesses often depend on phone-based interactions, a voice-first assistant is a realistic and valuable solution in this context.

1.5 SWOT Analysis

To evaluate the proposed Automated Voice Assistant System in a structured way, we performed a SWOT analysis. SWOT (Strengths, Weaknesses, Opportunities, Threats) is commonly used to identify internal factors (strengths and weaknesses) and external factors (opportunities and threats) that can influence the success of a solution. In this project, the SWOT analysis is particularly important because the system targets SMEs, which are highly sensitive to cost, reliability, and ease of adoption, and it will be implemented in the Republic of Moldova, where the business environment is dominated by SMEs [3].

The strengths represent internal advantages of the proposed solution, meaning features or design decisions that directly improve its usefulness for the target audience.

- *voice-first and 24/7 availability*. The system is designed to answer phone calls automatically and provide instant responses at any time. This directly addresses the most common SME problem: missed calls during peak hours or outside working hours;
- *designed specifically for SMEs (simplicity and fast deployment)*. Compared to enterprise platforms, the solution focuses on a limited number of high-value use cases (repetitive questions, call han-

ding, structured workflows). This reduces complexity and makes adoption more realistic for small businesses;

- *database-driven responses*. By connecting responses to a structured database, the assistant can provide up-to-date information such as working hours, service lists, and pricing. This increases reliability compared to static FAQ systems and reduces the risk of giving outdated information;
- *modular architecture and flexibility*. The system can combine telephony integration, speech-to-text, NLP logic, and text-to-speech as separate modules. This makes it easier to improve components independently (e.g., switching ASR models or optimizing TTS) without rewriting the entire system;
- *cost control through open-source and self-hosting options*. The design allows the use of open-source technologies (e.g., Whisper for speech recognition [6]) and self-hosted deployment (Docker-based). This can help SMEs keep long-term costs lower compared to strictly cloud-based solutions.

Weaknesses represent internal limitations or challenges that can reduce system effectiveness or increase implementation effort.

- *speech recognition limitations in real environments*. Phone audio quality, noise, accents, and fast speech can reduce recognition accuracy, especially in noisy SMEs (restaurants, shops). Although modern ASR (e.g., Whisper) is robust [6], errors are still unavoidable in live calls;
- *dependency on telephony infrastructure and connectivity*. The assistant requires stable internet and telephony integration. If connectivity is poor or the telephony provider has issues, the user experience can degrade;
- *initial business configuration effort*. Even with a simple dashboard, SMEs still need to provide correct data (hours, services, prices) and maintain it. If businesses do not update information, the assistant may become less trustworthy;
- *limited conversational coverage*. Since the solution is optimized for typical SME intents, it may not handle highly complex or unusual requests well. To avoid poor experiences, it must include good fallback and escalation strategies;
- *ongoing tuning and monitoring needs*. Real deployments often require iteration: improving intent rules, refining prompts, and analyzing call logs. SMEs may not have time for continuous tuning unless the system offers simple monitoring tools.

1.5.1 Market Opportunities, Threats, and System Benefits

Opportunities describe external factors that can increase adoption or open directions for expansion.

- *high SME presence in Moldova*. Since Moldova is dominated by SMEs [3], there is a broad potential user base for affordable automation tools that improve customer communication;
- *growing interest in digital transformation*. Moldova has active initiatives and policy directions

aimed at digitalization [4]. This environment can support openness to adopting automation in business processes, especially if the solution is simple and cost-effective;

- *expansion into additional workflows.* After a stable MVP, the system could be expanded to support more advanced SME tasks such as appointment scheduling with calendar sync, order placement, delivery coordination, or integration with CRM/accounting tools;
- *analytics as a value-added feature.* SMEs rarely have structured analytics about phone calls (peak times, most frequent questions, missed calls). Logging and reporting can become a strong additional benefit because it helps SMEs make operational decisions;
- *local market niche.* Many global enterprise platforms are expensive or complex for Moldovan SMEs. A localized, SME-oriented assistant can occupy a market niche by providing an affordable and practical alternative.

Threats represent external risks that could negatively affect the solution, even if the system is technically solid.

- *competition from major cloud providers.* Large platforms such as Google Dialogflow and Amazon Connect are continuously improving and could introduce simplified SME offerings over time [10, 11];
- *vendor pricing changes.* If the system relies on paid telephony minutes or external APIs, a change in pricing policies can affect operating costs. This is a general risk for solutions that use third-party providers;
- *privacy and compliance concerns.* Businesses may hesitate to adopt voice automation if they fear that customer calls are recorded or shared. The system must clearly define what data is stored and follow data protection principles under GDPR [1] and national Moldovan regulations [5];
- *low trust in automation for phone calls.* Some customers may prefer a human operator and may react negatively to automated systems. Poorly designed voice flows (long prompts, repeated misunderstandings) can lead to frustration and reputation damage for the business;
- *technical risks (latency and downtime).* Voice interactions are sensitive to delays and failures. If response latency is high or the system fails during calls, customers may hang up quickly, reducing perceived value.

As shown in Table 1.2, the SWOT analysis highlights that the main advantages of the proposed solution are its SME focus, availability, and cost control, while the key challenges involve ASR accuracy in real environments, adoption effort, and trust. The largest opportunities come from Moldova's SME-dominated market and ongoing digitalization trends, while the main threats relate to competition, vendor dependence, and privacy expectations.

Table 1.2 — SWOT analysis of the Automated Voice Assistant System

<i>Strengths</i>	<i>Weaknesses</i>
– Voice-first service with 24/7 availability. – SME-oriented design: simple and fast deployment. – Database-driven, up-to-date responses. – Modular architecture. – Cost control via open-source/self-hosting.	– Speech recognition errors in noisy calls. – Dependency on internet/telephony. – Requires maintained business data. – Limited coverage for complex requests. – Needs monitoring and tuning.
<i>Opportunities</i>	<i>Threats</i>
– Large SME base in Moldova [3]. – National digitalization initiatives [4]. – Expansion into appointments and CRM. – Call analytics as added value. – Affordable local SME niche.	– Competition from enterprise platforms [10, 11]. – Third-party pricing changes. – Privacy concerns [1, 5]. – Customer resistance to automation. – Latency and downtime risks.

The unique value proposition (UVP) of the proposed Automated Voice Assistant System can be summarized as follows:

The system provides small businesses in Moldova with an affordable, voice-first “virtual receptionist” that answers calls instantly, delivers consistent information based on the business database, and reduces missed opportunities—without the cost and complexity of enterprise contact-center platforms.

This value proposition is important because it directly connects the solution to the real constraints of SMEs described earlier: limited staff, peak-hour overload, high sensitivity to costs, and the need for practical deployment. Existing enterprise products such as Google Dialogflow and Amazon Connect offer powerful functionality, but they are typically designed for larger organizations, require technical setup, and can be expensive or difficult to maintain for micro and small businesses [10, 11]. The proposed system is positioned differently: it prioritizes *simplicity, reliability, and cost efficiency* for the SME environment.

The proposed solution delivers value in four main dimensions:

- *Immediate response and reduced missed calls* A major operational problem for SMEs is missed calls during busy hours. The assistant answers calls automatically, which means customers can obtain essential information immediately. This reduces the risk of losing customers who would otherwise call another business after not receiving an answer.
- *consistency and professionalism through standardized responses* In small teams, answers often differ depending on who responds and how busy they are. By using predefined conversation flows and database-driven responses (working hours, services, pricing), the system can deliver consistent information in a professional tone. This improves customer experience and reduces misunderstandings;

- *operational efficiency for SMEs* By automating repetitive call scenarios (e.g., opening hours, address, pricing), the solution reduces interruptions for staff members. This allows employees to focus on their core tasks (serving customers, delivering services), which is especially important in SMEs where staff work in multitasking roles;
- *cost-effective automation compared to enterprise platforms* The system is designed to be affordable for SMEs and to scale gradually. It uses a modular architecture and can run in containerized deployment (Docker), which supports portability and cost control. Telephony integration can be implemented through Twilio for rapid deployment and prototyping [7], while keeping the option of future cost optimization through alternative telephony solutions if needed. In addition, using modern open speech recognition models such as Whisper [6] can reduce reliance on expensive proprietary services.

This domain analysis highlighted that customer communication in SMEs is strongly affected by limited staff, peak-hour overload, and the high frequency of repetitive questions. Since many SMEs depend on phone calls as a primary interaction channel, missed calls and delayed responses can translate directly into lost customers and reduced satisfaction. The Moldovan context reinforces the relevance of this problem, because the local economy is dominated by SMEs [3], and many businesses operate with minimal administrative support.

We also reviewed existing market solutions such as Google Dialogflow, Amazon Connect, and Twilio Flex/Autopilot [10, 11, 12, 13]. Although these platforms are powerful and mature, they are often too complex or costly for micro and small enterprises, especially when technical setup and long-term maintenance are considered. Based on the SWOT analysis and the defined unique value proposition, the proposed Automated Voice Assistant System is positioned as a more suitable option for this specific case: it focuses on voice-first automation, database-driven correctness, cost control, and practical adoption for SMEs.

Finally, the SMART objectives provide measurable targets that connect the problem definition to evaluation: reducing response time, improving customer satisfaction, and reducing missed calls. Together, these elements justify the development of the system and establish clear requirements and success criteria for the next stages of the project.

2 Functional Requirements

This section presents the system requirements for the *Automated Voice Assistant System for Small Businesses*. In software engineering, requirements are commonly divided into two complementary categories: *functional* and *non-functional*. Functional requirements specify *what* the system must do, meaning the concrete services, behaviors, and interactions that the system provides to its users. In contrast, non-functional requirements specify *how well* the system must perform these functions and under what constraints it must operate, covering quality attributes such as performance, availability, usability, security, privacy, scalability, and maintainability.

This separation is particularly important for an automated voice assistant used in real customer interactions. The system is expected not only to answer calls and provide correct information, but also to do so in a way that is fast, reliable, understandable, and secure. Because the application may process personal data (e.g., caller identifiers and call metadata), compliance with applicable data protection principles must also be considered. Together, the functional and non-functional requirements define the baseline scope of the project and provide clear criteria for implementation, testing, and validation of the final solution.

For clarity and verifiability, each requirement is presented using a consistent structure:

- *description*: brief explanation of the capability and its purpose;
- *requirement statement*: a normative “shall” statement;
- *notes / acceptance criteria examples*: practical conditions used to confirm correct implementation.

2.1 Functional Requirements Specification

- requirement 1 — Automatic Call Answering and Greeting;

Description: Ensure that incoming calls are not missed and that callers receive an immediate, professional first response. This establishes the assistant as the first point of contact.

Requirement statement: The system shall automatically answer incoming customer calls and deliver an initial greeting message that introduces the assistant and prompts the caller to state their request.

Notes / acceptance criteria examples:

- the assistant answers calls without human intervention;
 - the greeting includes a short prompt (e.g., “How can I help you?”);
 - the greeting text should be configurable by the business owner.
- requirement 2 — Speech Recognition (Speech-to-Text);

Description: Convert the caller’s spoken input into text so it can be processed by the language understanding component.

Requirement statement: The system shall convert the caller's spoken input into text using a speech-to-text mechanism.

Notes / acceptance criteria examples:

- the transcription output is forwarded to the intent analysis module;
 - if transcription confidence is low, the system requests repetition or clarification;
 - the solution should support Romanian and Russian speech as required by project objectives.
- requirement 3 — Intent Identification and Request Classification;

Description: Determine what the caller is requesting (intent) based on the transcribed text.

Requirement statement: The system shall analyze the transcribed text and identify the caller's intent, including (but not limited to) requests related to working hours, location, services offered, pricing, availability, and contact details.

Notes / acceptance criteria examples:

- typical questions (e.g., “When do you close?”) are mapped to the correct intent category;
 - the intent list can be extended to support additional business needs;
 - the system supports multilingual intent recognition for Romanian and Russian.
- requirement 4 — Retrieval of Business Information (Knowledge Base / FAQ);

Description: Provide answers based on the most recent business-configured information stored in the system.

Requirement statement: The system shall retrieve relevant response content from the business knowledge base (configured FAQs and business information stored in the database) based on the identified intent.

Notes / acceptance criteria examples:

- if the owner updates hours/prices in the dashboard, the assistant uses the updated values in subsequent calls.
 - responses may be implemented as templates with structured fields (e.g., open/close times).
 - the system prioritizes official business data to ensure consistency.
- requirement 5 — Response Formulation and Dialogue Continuation;

Description: Provide an appropriate response and, when required, ask follow-up questions to obtain missing details.

Requirement statement: The system shall formulate a response aligned with the caller's intent and, when required, request additional information to complete the interaction.

Notes / acceptance criteria examples:

- if the customer asks for pricing but multiple services exist, the assistant requests the specific service.

- responses remain concise and suitable for phone interactions.
 - the system avoids fabricating information and uses safe fallback behavior when uncertain.
- requirement 6 — Spoken Response Delivery (Text-to-Speech);
- Description:* Deliver the selected response in audio form so the caller receives the information naturally during the phone conversation.
- Requirement statement:* The system shall convert the selected textual response into spoken audio using text-to-speech synthesis and play the response to the caller within the active call session.
- Notes / acceptance criteria examples:*
- spoken output is intelligible and appropriate for a business setting;
 - the response language matches the caller’s language where supported;
 - audio playback is not clipped and completes successfully for typical response lengths.
- requirement 7 — Handling Unrecognized or Complex Requests (Fallback / Escalation);
- Description:* Ensure appropriate behavior for ambiguous or unsupported requests to reduce caller frustration and prevent incorrect information delivery.
- Requirement statement:* The system shall detect unrecognized, ambiguous, or unsupported requests and respond using a fallback strategy that includes clarification prompts and/or escalation (redirection to the business owner or storing the request for follow-up).
- Notes / acceptance criteria examples:*
- if intent confidence is below a defined threshold, the assistant asks clarifying questions;
 - after a limited number of failed attempts, the assistant redirects the call or records the request for follow-up;
 - escalation outcomes are recorded in call logs.
- requirement 8 — Call Logging and Interaction Metadata Storage;
- Description:* Store structured interaction data for analytics, troubleshooting, and improvement of the assistant.
- Requirement statement:* The system shall record and store metadata for each call, including at minimum: date and time, call duration, identified intent (or request category), interaction outcome (answered, redirected, unresolved), and customer feedback if provided.
- Notes / acceptance criteria examples:*
- each call creates one log record in the database;
 - logs support filtering by date, intent, and outcome;
 - sensitive data storage is minimized and handled according to privacy requirements.
- requirement 9 — Customer Feedback Collection;
- Description:* Gather customer feedback to evaluate service quality and measure improvement over

time.

Requirement statement: The system shall request and store customer feedback at the end of the call, using a method such as a numeric rating (1–5) and/or short verbal feedback.

Notes / acceptance criteria examples:

- feedback is linked to the corresponding call log entry;
- the owner can view aggregated satisfaction metrics in the dashboard.
- requirement 10 — Business Owner Management Dashboard (Configuration);

Description: Enable SMEs to maintain and update assistant knowledge without technical support, keeping business information accurate over time.

Requirement statement: The system shall provide a secure web-based management dashboard that allows authorized business owners to add, edit, and remove FAQs and business information used by the assistant.

Notes / acceptance criteria examples:

- the owner can update working hours, address, services, and pricing notes;
- updates are persisted and applied in subsequent interactions without redeployment;
- access is restricted to authorized users (authentication).
- requirement 11 — Analytics and Reporting

Description: Provide reporting to help businesses understand usage, peak times, and customer needs, supporting operational decision-making.

Requirement statement: The system shall generate daily and monthly reports showing the number of calls handled, common questions/intents, missed or redirected calls, and satisfaction levels.

Notes / acceptance criteria examples:

- reports answer questions such as “Which topics are most frequent?” and “When are peak call times?”;
- reports are displayed in the dashboard and may be exportable (e.g., CSV/PDF).
- requirement 12 — Support for Continuous Improvement;

Description: Support periodic improvement of the intent recognition and response quality using interaction logs and feedback, while respecting privacy constraints.

Requirement statement: The system shall support periodic improvement of the language understanding component by using stored interaction data (e.g., intent labels, unresolved queries, and feedback) to refine model accuracy over time, subject to privacy constraints.

Notes / acceptance criteria examples:

- unresolved calls can be reviewed and converted into training examples;
- training data usage can be controlled/approved by the owner or administrator;

- model updates are versioned to support rollback if quality decreases.

2.2 Non-functional Requirements Specification

- requirement 1 — Performance (Response Time); *Description:* In a live phone conversation, long delays reduce usability and make the interaction feel unnatural. The system should respond quickly enough to maintain a smooth dialogue and avoid caller frustration.

Requirement statement: The system shall process and respond to a caller request within 2 seconds under normal operating conditions.

Notes / acceptance criteria examples:

- “response time” is measured from the end of user speech to the start of audio playback;
 - performance tests should be executed using realistic audio samples and typical load;
 - if external APIs are used, the system should implement timeouts and fallback behavior.
- requirement 2 — Availability (24/7 Operation);

Description: One of the key values of the assistant is constant availability, especially outside working hours or during peak periods.

Requirement statement: The system shall maintain at least 99% uptime to support continuous 24/7 operation.

Notes / acceptance criteria examples:

- uptime is evaluated monthly based on monitoring data;
 - planned maintenance windows should be announced and minimized;
 - the system should implement health checks and automated restart mechanisms.
- requirement 3 — Reliability and Fault Tolerance;
- Description:* The system must behave predictably and avoid failures during calls. When failures do occur (e.g., STT errors, network instability), the system should fail gracefully instead of dropping calls without explanation.

Requirement statement: The system shall handle component errors gracefully by applying fallback strategies (clarification prompts, escalation, or safe termination) without corrupting stored data.

Notes / acceptance criteria examples:

- if the STT service fails, the system informs the caller and escalates or retries;
 - partial logs are still stored for troubleshooting (without violating privacy rules);
 - database operations should use transactions for consistency.
- requirement 4 — Usability (Non-technical Operation);
- Description:* SME owners typically have moderate digital literacy and limited time. The dashboard must be intuitive and require minimal training to configure FAQs and view reports.

Requirement statement: The management dashboard shall be usable by non-technical users and support common configuration tasks (editing FAQs, updating hours, viewing reports) without developer assistance.

Notes / acceptance criteria examples:

- a new user should be able to update business hours in under 5 minutes;
 - the interface should provide clear labels, validation messages, and help text;
 - role-based access should ensure owners only see relevant options.
- requirement 5 — Security (Data Protection and Access Control);

Description: The system stores business data and call logs and may handle personally identifiable information (PII). Unauthorized access could expose customer data and damage business trust.

Requirement statement: The system shall restrict access to the management dashboard and stored data using authenticated user accounts and role-based authorization.

Notes / acceptance criteria examples:

- Passwords are stored using secure hashing (not plain text).
 - Administrative endpoints are not accessible without authorization.
 - Audit logs should record critical administrative actions (e.g., login events, FAQ edits).
- requirement 6 — Encryption of Sensitive Data;

Description: Stored call logs and business data may contain sensitive information and must be protected at rest and in transit. Encryption is a practical baseline control. NIST recognizes encryption as a key method for confidentiality protection [**nist80053**].

Requirement statement: The system shall encrypt sensitive stored data and call logs using AES-256 (or an equivalent industry-standard mechanism) and shall use TLS for data in transit.

Notes / acceptance criteria examples:

- database storage uses encryption-at-rest where supported by the hosting environment;
 - all dashboard connections use HTTPS (TLS);
 - encryption keys are managed securely (not hard-coded in source code).
- requirement 7 — Privacy and Compliance (GDPR);

Description: The system processes personal data (e.g., phone numbers and interaction metadata). GDPR requires lawful processing, transparency, purpose limitation, and data minimization [**gdpr2016**].

Requirement statement: The system shall support GDPR-aligned processing by minimizing stored personal data, enabling data retention policies, and providing mechanisms to delete or anonymize personal data upon request.

Notes / acceptance criteria examples:

- configurable retention period for call logs (e.g., 30/90/180 days);

- option to store only metadata by default and avoid storing full audio/transcripts unless necessary;
 - ability to delete a customer’s data when legally required (subject to business policy).
- requirement 8 — Scalability (Concurrent Calls);

Description: Although SMEs are small, the system must still handle peak loads (e.g., lunch rush for food services). Scalability ensures that the system remains responsive under increased call volume.

Requirement statement: The system shall support up to 100 concurrent calls while maintaining the defined performance target under expected operating conditions.

Notes / acceptance criteria examples:

- load tests simulate concurrent calls with realistic speech inputs;
 - the architecture should allow horizontal scaling of stateless components;
 - resource limits and autoscaling policies should be documented.
- requirement 9 — Maintainability and Modularity;

Description: The system is expected to evolve (new intents, languages, integrations). Maintainability reduces long-term cost and supports safe updates. ISO/IEC 25010 defines maintainability as a key software quality characteristic [iso25010].

Requirement statement: The system shall be designed in a modular manner so that AI components, backend services, and dashboard functionality can be updated or replaced with minimal impact on other components.

Notes / acceptance criteria examples:

- speech recognition, NLP, and TTS components are accessed through well-defined interfaces;
 - updates can be deployed using containerization (e.g., Docker) with version control;
 - changes to FAQ content do not require redeployment of the backend.
- requirement 10 — Monitoring and Operational Observability.

Description: Continuous operation requires visibility into system health and performance. Monitoring enables early detection of failures and supports uptime goals.

Requirement statement: The system shall provide monitoring capabilities, including health checks and operational metrics (e.g., call volume, error rates, response latency).

Notes / acceptance criteria examples:

- health endpoints report availability of core services (API, database, STT/NLP/TTS);
- metrics are collected and visualized for administrators;
- alerts can be configured for critical thresholds (e.g., high error rate, downtime).

3 System Design

3.1 System Design

The project architecture is modeled using UML diagrams in order to describe both the functional behavior and the structural design of the system. Since the solution is composed of two major subsystems, the diagrams in this chapter are organized accordingly:

- *business web portal (Dashboard)* – focused on configuration, business data management, and monitoring/analytics;
- *voice assistant runtime* – focused on real-time phone call handling, speech processing, response delivery, and escalation.

Although these subsystems are described separately for clarity, they share common components such as the database and configuration data, and they form a single integrated system.

Figure 3.1 presents the general architecture of the full solution and how the two subsystems (Voice Assistant Backend and Business Web Portal) interact with external services and the shared database. The goal of this architecture is to support real-time phone conversations while still being configurable by the business owner through a web dashboard.

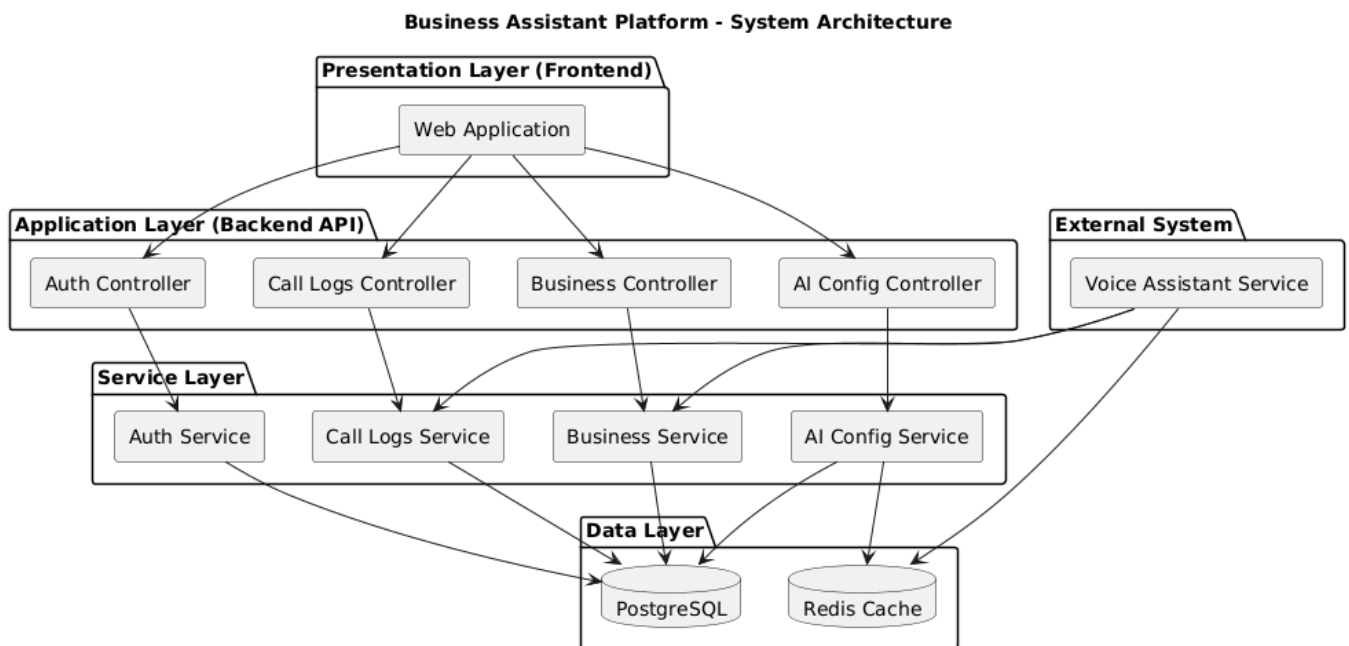


Figure 3.1 - General system architecture of the Voice Assistant and Business Web Portal

On the *voice assistant side*, the customer interacts with the system through a phone call handled by *Twilio Telephony*. Twilio forwards call events to the *Call Handler (Webhooks)*, which acts as the entry point of the backend. During the call, the Call Handler streams audio to the *STT (Speech-to-Text)* service to obtain transcripts. These transcripts are then sent as an intent request to the *NLP / Intent Engine*, which

determines what the customer wants and produces the response text. If the user request is more complex or the intent confidence is low, the Call Handler triggers the *Escalation Logic*, which can alert the business owner via *Notifications (SMS/Email)* or initiate a call transfer when needed. Once a response is ready, it is sent to the *TTS (Text-to-Speech)* service and played back to the customer through Twilio.

The *PostgreSQL database* is a shared component used by both subsystems. It stores business configuration and content (e.g., FAQs and business profile data), as well as operational data such as call logs. The *Logging Service* in the voice assistant backend records important information from each call (intent, timestamps, outcomes) so that the system can be monitored and improved. For appointment booking, the NLP/Intent Engine can optionally communicate with *Google Calendar* to create events and confirm bookings.

On the *business web portal side*, the business owner uses the *Dashboard UI* to manage the system and view analytics. The UI communicates over HTTPS with the *Portal API*, which reads and writes the same PostgreSQL data used by the assistant. This makes the assistant configurable without redeploying the call-handling system, since updates to FAQs, business information, and settings become available immediately through the shared database.

3.2 Use Case Diagrams

This section presents the UML diagrams that describe the behavior and structure of the *business web portal*. The portal provides the business owner/admin with the ability to configure the assistant and to monitor call data and analytics.

Use Case Diagram 1 – Configuration Management (Profile, Hours, Services, Greeting, Escalation)

Figure 3.2 describes the core configuration functionality of the *Business Web Portal (Dashboard)*. The primary actor is the *Business Owner*, who can manage all assistant-related configuration that must be stored reliably in the system. The *Administrator* actor has more limited access and is mainly able to view configuration summaries (read-only monitoring role).

The main use cases include:

- *manage business profile* – updating business identity and contact information (name, address, phone);
- *manage working hours* – defining weekly schedule rules that will later be used by the assistant when answering availability-related questions;
- *manage services & pricing* – maintaining a list of services/products and their prices so responses remain consistent and up-to-date;
- *configure assistant greeting* – setting the message played at the beginning of a call;
- *configure escalation / forwarding* – defining conditions and the forwarding number for human escalation;

- *view configuration summary* – verifying the current settings that are active for the business.

Two shared system behaviors are modeled using <<include>> relations:

- *Validate Configuration Data* is included by all configuration actions to ensure data integrity (e.g., mandatory fields, valid phone number format, schedule consistency, price format).
- *Save Settings to Database* is included to represent the persistence step after successful validation, ensuring the assistant runtime always reads a consistent configuration.

Overall, this use case diagram supports the portal requirements related to business data management and assistant configuration, and it emphasizes the importance of validation and reliable storage before the settings become active.

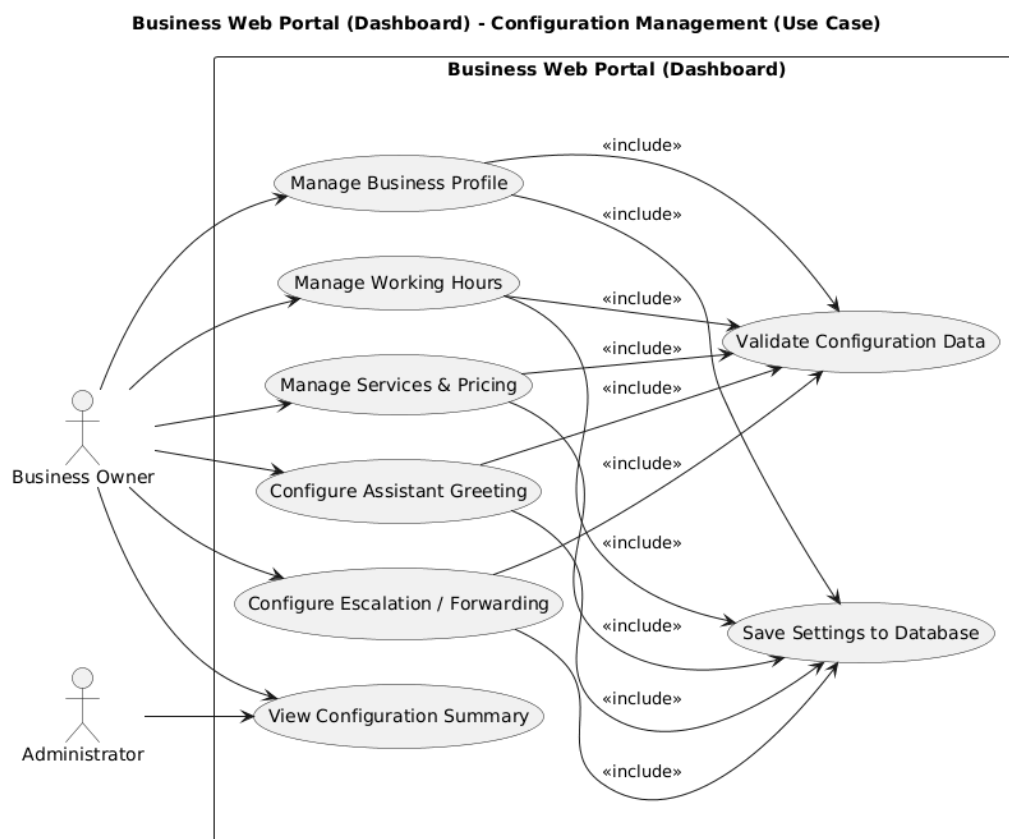


Figure 3.2 - Use Case Diagram 1 – Configuration Management (Business Web Portal)

Use Case Diagram 2 – Dashboard Main Functions (Calls, Profile, Calendar, Availability, FAQs, Analytics) Figure 3.3 presents a higher-level view of the dashboard features accessible after authentication. The main actor is the *Business Owner*, who uses the portal to configure the assistant and to monitor performance. The *Administrator* role is also present and can access a subset of monitoring-related features (e.g., analytics).

The diagram highlights the following user goals:

- *view calls* – viewing call records handled by the system (history and outcomes);
- *configure Business Profile* – editing core business information shown in assistant answers;

- *edit FAQs* – maintaining the knowledge base used for answering common customer questions;
- *view analytics* – opening aggregated metrics (call volume, peaks, resolution vs escalation);
- *integrate Google Calendar* with the included use case *Manage Availability* – representing the connection between calendar integration and availability management, where availability is derived from calendar events and/or configured schedules.

The <<include>> relationship between *Integrate Google Calendar* and *Manage Availability* shows that availability management depends on successful calendar integration (OAuth connection, calendar selection, token storage). This diagram therefore represents the main dashboard entry points and how configuration and monitoring features are grouped for non-technical users.

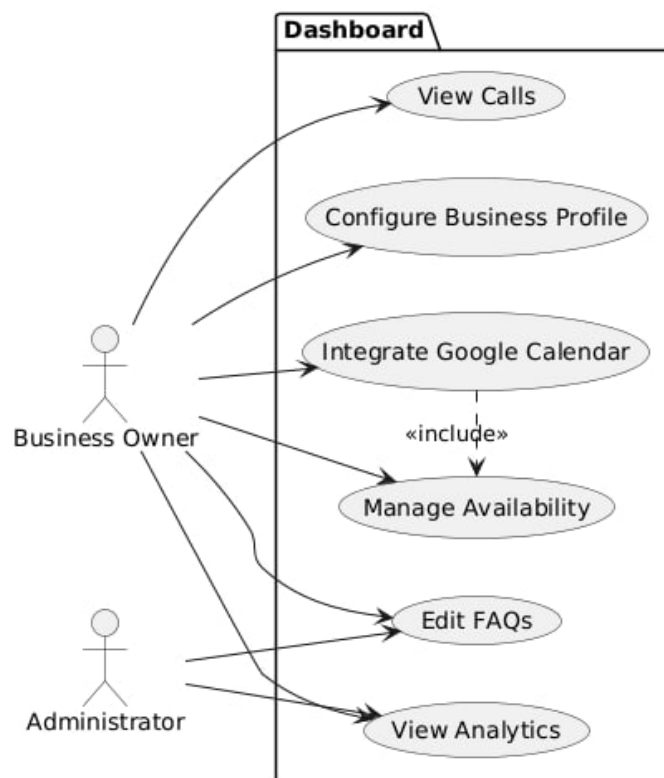


Figure 3.3 - Use Case Diagram 2 – Dashboard Main Functions

Use Case Diagram 3 – Knowledge Base Management (FAQ CRUD and Sync)

Figure 3.4 focuses on the *knowledge base management* functions available in the dashboard. The *Business Owner* is responsible for maintaining the FAQ entries that represent frequently asked customer questions and ensuring that the information provided by the system remains accurate, up to date, and aligned with real business operations. This is particularly important in SMEs, where working hours, prices, and service availability can change frequently and must be reflected immediately in the assistant’s responses.

The diagram includes standard CRUD operations:

- *add FAQ*;
- *edit FAQ*;

– *delete FAQ*.

Additionally, a *Sync Knowledge Base* use case is included to represent scenarios where the system needs to synchronize, refresh, or rebuild knowledge base indexes. This may occur after multiple updates, after bulk imports of FAQ data, or after modifications to retrieval and matching logic that affect how user queries are processed. The synchronization process ensures that the assistant always uses the latest version of the business knowledge without inconsistencies between stored data and runtime behavior.

This synchronization is handled by a dedicated *Knowledge Base Service*, which is responsible for updating and maintaining records in the database, as well as ensuring that any indexing structures or cached representations remain consistent after changes. In more advanced implementations, this service may also handle validation rules, duplicate detection, or semantic re-indexing of FAQ entries.

The <<include>> relations from the business owner to the CRUD actions emphasize that these operations are an integral part of the overall knowledge base management process, and that every modification is immediately persisted and propagated to the system layer. This guarantees that updates are not only stored but also reflected in the assistant's runtime behavior.

Overall, this use case diagram supports the system goal of maintaining consistent, reliable, and business-controlled responses, ensuring that the assistant always reflects the current state of the SME's operational knowledge.

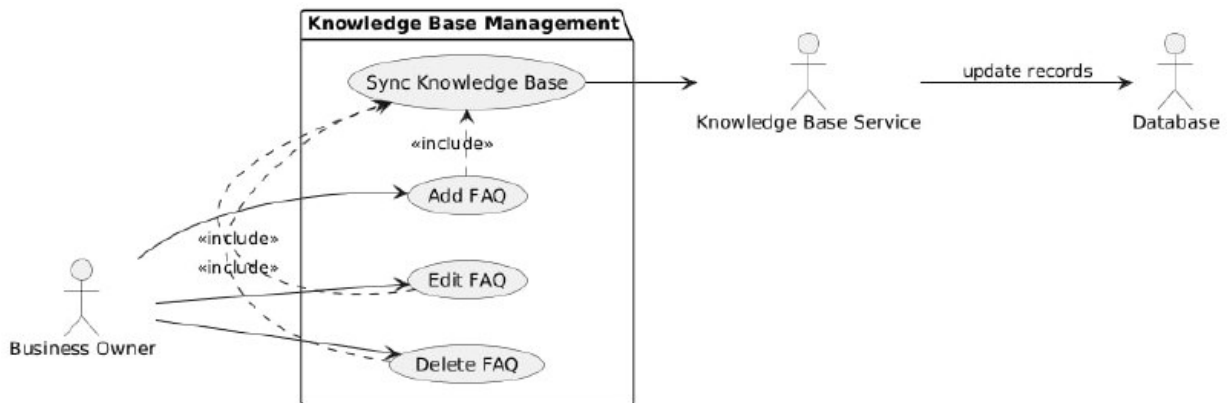


Figure 3.4 - Use Case Diagram 3 – Knowledge Base Management

Use Case Diagram 4 – Authentication & Account System Figure 3.5 presents the *authentication and account management* use cases of the Business Web Portal. The main actor is the *Business Owner*, who must authenticate before accessing any configuration or monitoring functions of the dashboard. The diagram also includes supporting external actors/services: an *Authentication Service* (responsible for credential checks, session handling, and security policies) and an *Email Provider* (used for verification and password recovery flows).

The account-related use cases include:

3.3 Sequence Diagrams

Sequence Diagram 1 – Business Owner Data Setup Flow Figure 3.6 illustrates the initial setup and configuration flow performed by the *Business Owner* through the dashboard. The interaction begins when the owner accesses the portal (login/registration). The *React Dashboard (Web UI)* sends an authentication request to the *FastAPI Backend*, which validates the credentials/session and returns a successful response that allows the dashboard to load.

After authentication, the owner enters the main business data required by the system (for example: working hours, address, initial FAQ entries, and optional calendar-related information). The Web UI submits this configuration through the backend API. The backend performs validation and then persists the information in the *PostgreSQL Database*. Once the database confirms that the configuration has been stored successfully, the backend returns a success response to the UI, and the dashboard displays a confirmation message (e.g., “Data saved successfully”).

Finally, the owner may request to view the currently stored configuration. In this step, the Web UI triggers a fetch request to the backend. The backend retrieves the stored business information from the database and returns it to the UI so that the owner can verify the saved state. This sequence demonstrates the core principle of the portal: configuration is edited by the business user, validated by the backend, and stored centrally so that it can later be used consistently by other subsystems.

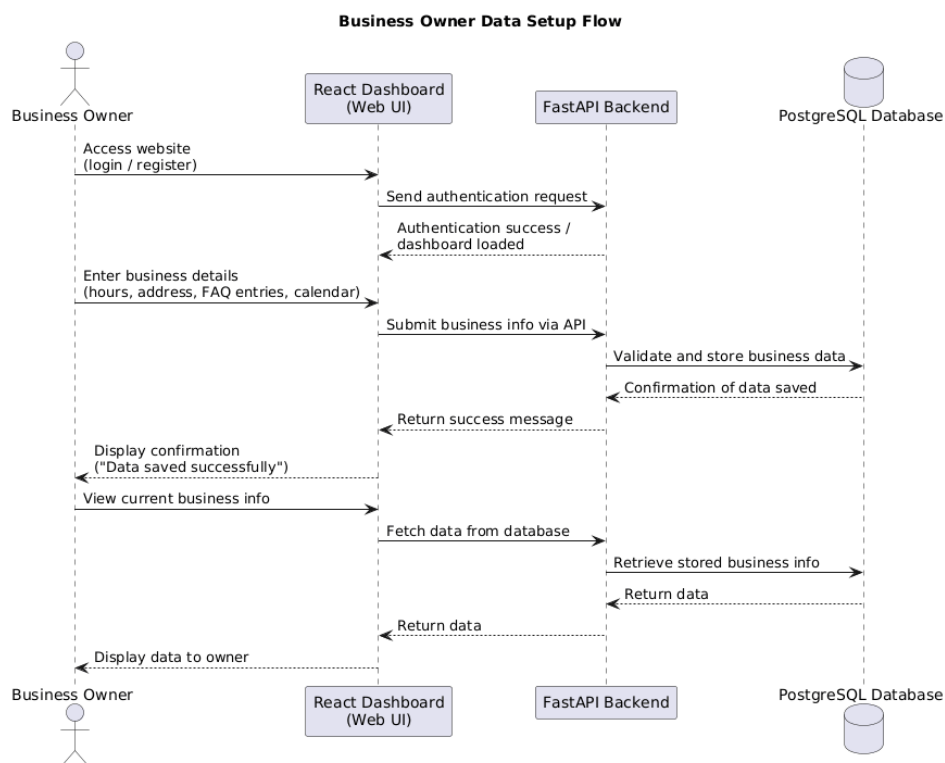


Figure 3.6 - Sequence Diagram 1 – Business Owner Data Setup Flow

Sequence Diagram 2 – FAQ Management (Create/Edit/Delete) Figure 3.7 describes how the business owner maintains the FAQ knowledge base through the dashboard. First, the owner opens the *FAQs* page in the React web interface. The UI sends an authenticated GET `/faqs` request to the FastAPI backend, which queries the PostgreSQL database for FAQ entries associated with the current business (`business_id`) and returns the list to the UI for display.

The diagram then shows three CRUD scenarios:

- *create new FAQ*: the owner submits a question and answer. The backend validates input (authentication and required fields) and inserts a new record into the database. A 201 Created response confirms that the new FAQ has been stored;
- *edit existing FAQ*: the owner updates an FAQ entry. The backend validates permissions (ensuring the user belongs to the same business and has sufficient rights) and updates the corresponding database record. A 200 OK response confirms the update;
- *delete FAQ*: the owner deletes an entry. The backend confirms permissions and checks referential constraints, then removes the record. A 204 No Content response indicates successful deletion, and the UI removes the item from the visible list.

This sequence is important because it demonstrates the dashboard’s controlled mechanism for maintaining the assistant’s knowledge base: all modifications go through authenticated API calls, are validated by the backend, and are persisted centrally so the runtime assistant can use the latest FAQ data.

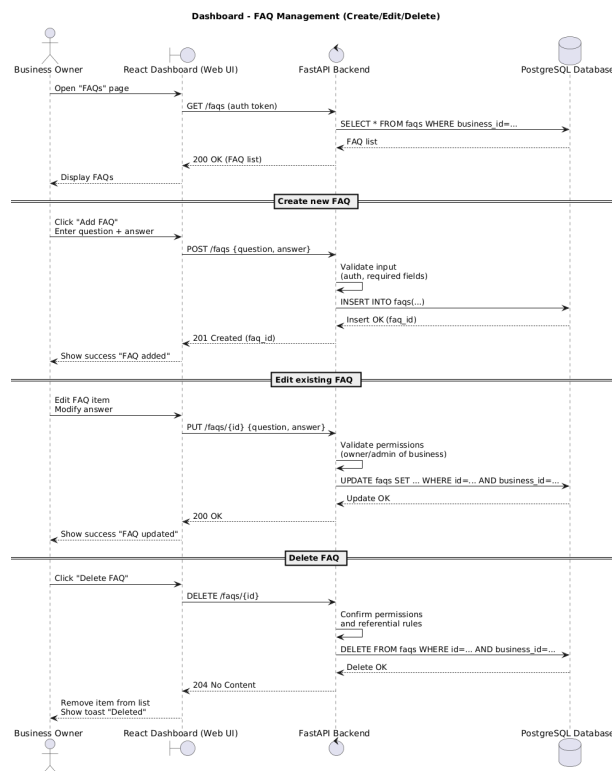


Figure 3.7 - Sequence Diagram 2 – FAQ Management (Create/Edit/Delete)

Sequence Diagram 3 – Call Logs: List, Filters, Details, Export (CSV) Figure 3.8 describes how the business owner reviews historical call data in the dashboard and optionally exports results. The sequence starts when the owner opens the *Call Logs* page. The React UI sends an authenticated request such as GET /calls with filter parameters (time range, status/outcome, pagination). The FastAPI backend authorizes the request by extracting the business_id from the session/token and then queries the PostgreSQL database using the provided filters. The result includes call rows and total count information so the UI can render pagination.

Next, the diagram shows the *call details* scenario. When the owner clicks a specific call entry, the UI requests GET /calls/{call_id}. The backend retrieves:

- call metadata (timestamps, duration, outcome);
- ordered event/intents history (fallbacks, escalation decisions, errors).

The UI then renders a timeline view that provides transparency into what happened during the call and how the assistant behaved.

Finally, the diagram includes the optional export feature (aligned with FR-WP-12). When the owner clicks *Export CSV*, the UI requests GET /calls/export.csv using the same filters. The backend again authorizes the request and validates export limits (e.g., maximum date range or row count), queries the database, generates the CSV file stream, and returns it for download. This supports offline reporting and business analysis.

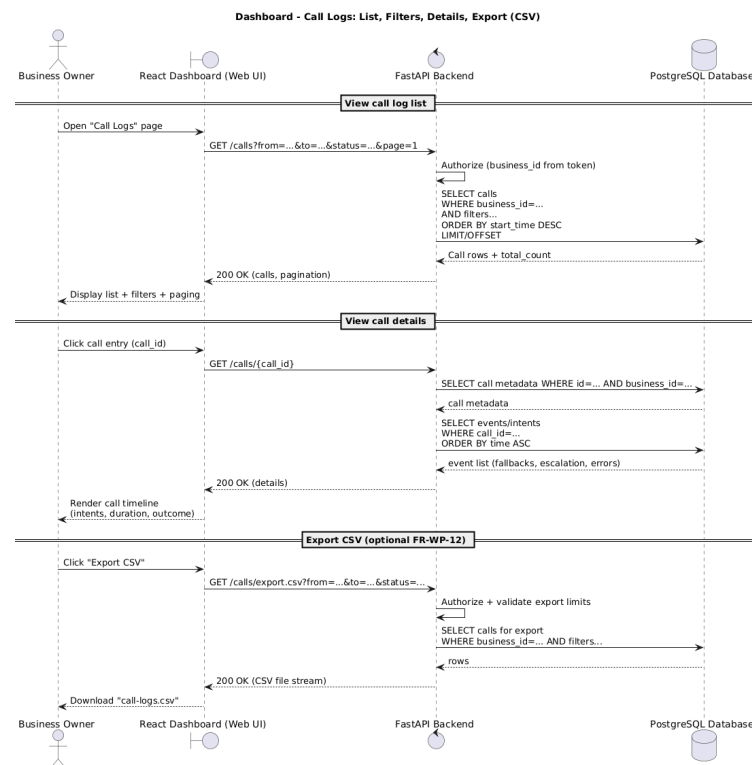


Figure 3.8 - Sequence Diagram 4 – Call Logs: List, Filters, Details, Export (CSV)

3.4 Activity Diagrams Collaboration Diagram

Activity Diagram 1 – System Configuration & Integration Setup Workflow Figure 3.9 describes the onboarding and integration setup process performed through the dashboard. The workflow begins when the owner creates an account or selects an existing business profile, then opens the settings/integrations section. The owner configures basic business data (business name, phone numbers, timezone) which is required for correct assistant behavior and correct timestamp/availability computations.

Next, the owner configures telephony integration (for example mapping a Twilio phone number and providing webhook URL/credentials). The diagram includes a decision point to validate the telephony configuration. If validation fails, the dashboard shows a configuration error and requests correction. If validation succeeds, the system saves the telephony settings in the backend configuration store.

Optionally, the owner can connect Google Calendar for booking. If the owner chooses to connect, the dashboard starts an OAuth flow with Google, receives an access token and calendar identifier, then stores tokens securely and links the integration to the business profile. After integration steps are complete, the owner configures assistant behavior settings (greeting, language, escalation rules, business hours). The workflow ends when the full configuration is saved to the database and the system triggers a reload of runtime configuration so the assistant can use the latest settings. Finally, the dashboard displays a success message and readiness status.

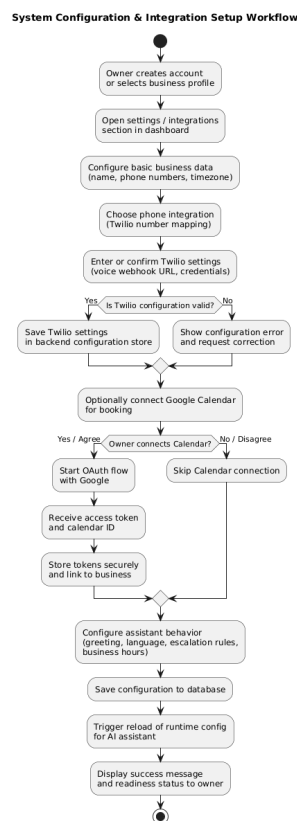


Figure 3.9 - Activity Diagram 1 – System Configuration & Integration Setup Workflow

Activity Diagram 2 – Business Owner Management & Dashboard Interaction Figure 3.10 summarizes the general interaction flow between the business owner and the dashboard for routine management tasks. The process starts with logging in and authenticating through the backend. A decision node checks whether authentication is successful. If authentication fails, the dashboard displays an error message and the workflow ends without granting access.

When authentication is successful, the dashboard loads the main interface and the owner can perform configuration and monitoring actions. The diagram highlights typical tasks such as adding/editing FAQ entries, updating business hours or profile information, and reviewing analytics. After the owner makes changes, the backend persists updates to the database. The final activity is showing confirmation to the owner, including either a success notification (“data saved successfully”) or updated metrics on the dashboard. This workflow captures the portal’s primary purpose: enabling non-technical users to manage assistant data and monitor system usage through a simple web interface.

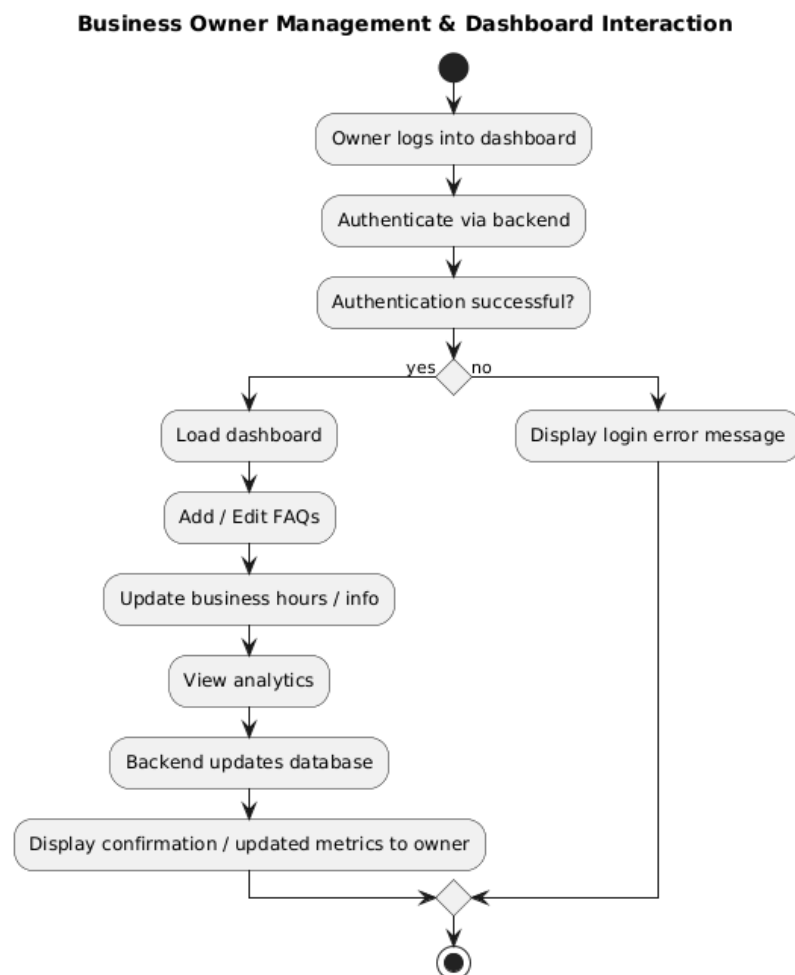


Figure 3.10 - Activity Diagram 2 – Business Owner Management & Dashboard Interaction

3.5 State Diagrams Class Diagrams

State Diagram 1 – Composite States: Business Configuration Editing Lifecycle

Figure 3.11 shows a state diagram with *composite states*, focusing on the configuration editing lifecycle inside the dashboard. The outer flow begins in *ViewingDashboard*. When the user opens settings, the system transitions to the composite state *EditingConfiguration*, which contains several internal substates and transitions that represent the detailed editing process. This composite structure is used to group related configuration activities into a single high-level state while still preserving detailed behavioral logic inside it.

Inside *EditingConfiguration*, the user first reaches *ChooseSection*, where a configuration category is selected based on the type of data that needs to be modified. The diagram models multiple editing substates such as *EditingFAQs*, *EditingProfile*, *EditingWorkingHours*, and *EditingServicesPricing*. Each of these substates represents a specific domain of business configuration and ensures that changes remain logically separated and easier to validate. When the user clicks save, the workflow transitions to *ValidateChanges*, where all modifications are checked for correctness, completeness, and consistency with system constraints.

If validation fails, the system enters *ValidationError*, and the user is informed about incorrect or missing fields. The user can then correct the input and return to the appropriate editing section without losing progress. If validation succeeds, the system proceeds to *Saving*, which commits changes to the database and ensures persistence of the updated configuration.

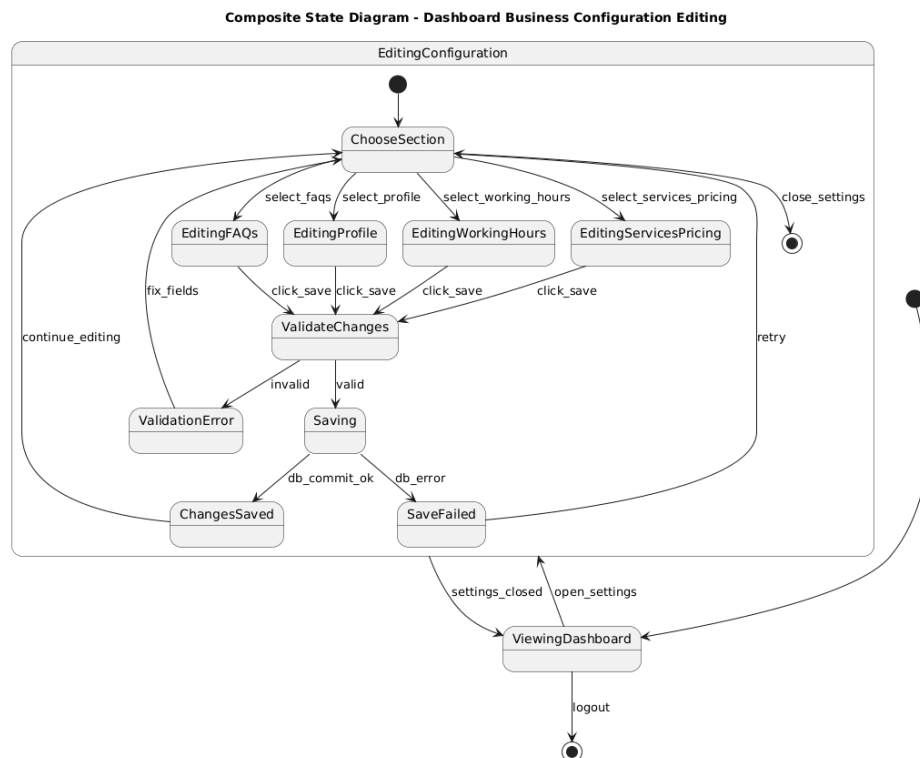


Figure 3.11 - State Diagram 1 – Business Configuration Editing (Composite States)

Class Diagram – Business Web Portal (Dashboard) Figure 3.12 presents the class-level structure of the *Business Web Portal (Dashboard)* subsystem. The diagram focuses on the main controller/view responsibilities, the editing modules used for configuration management, and the shared business context that encapsulates the data displayed and modified in the portal.

At the interaction layer, the interface `IUserInput` represents user-driven actions in the UI such as selecting menu items, editing form fields, and submitting changes. The `DashboardView` realizes this interaction by providing methods to render the UI and display feedback messages to the user, while also tracking the `currentSection` currently displayed.

The `DashboardController` coordinates the overall dashboard behavior. It maintains the current authenticated user identifier (`currentUserId`) and exposes operations such as opening the dashboard and saving changes. The controller uses specialized editor/viewer components to implement specific dashboard functions:

- `faqEditor` manages FAQ CRUD operations (`addFaq`, `editFaq`, `removeFaq`);
- `businessInfoEditor` updates core configuration such as opening hours and contact details;
- `analyticsViewer` loads and displays reports and analytics results.

Session-related behavior is modeled using `DashboardSessionManager`, which manages the session lifecycle (`startSession`, `endSession`) and holds session metadata such as `sessionId` and `status`. The multiplicity indicates that the controller can manage multiple session instances over time, while the controller remains a single coordination point within a running UI instance.

A key concept in the diagram is `BusinessContext`, which acts as a shared container for business-scoped data used across the dashboard. It holds both structured profile/configuration information (`profile`) and knowledge base data (`faqData`), and provides a `load()` operation to retrieve current state. The context aggregates domain objects:

- `faqEntry (0..*)` representing stored FAQ items with `id`, `question`, and `answer`;
- `report (0..*)` representing analytics/report outputs with `id`, `title`, and `generatedAt`.

The saving step has two possible outcomes: *ChangesSaved* when the database commit succeeds, or *SaveFailed* when an unexpected error occurs during persistence (for example connectivity issues or database constraints). In the failure case, the diagram provides a retry path back into the editing flow, allowing the user to attempt saving again after resolving the issue. This improves system resilience and user experience by avoiding data loss.

Finally, persistence/loading dependencies are modeled through the `IDataSource` interface, which abstracts the underlying data store access. `BusinessContext` relies on `IDataSource.load()` to populate its in-memory representation of configuration and analytics data. This separation supports maintainability: UI components and editors operate on the business context, while the data source implementation can be

swapped or extended (e.g., database, cache, or external service) without changing higher-level dashboard logic.

Overall, this class diagram demonstrates a clear separation of concerns: the controller orchestrates actions, specialized components handle domain-specific operations (FAQs, business info, analytics), the view renders UI feedback, and the business context centralizes the data needed by the dashboard.

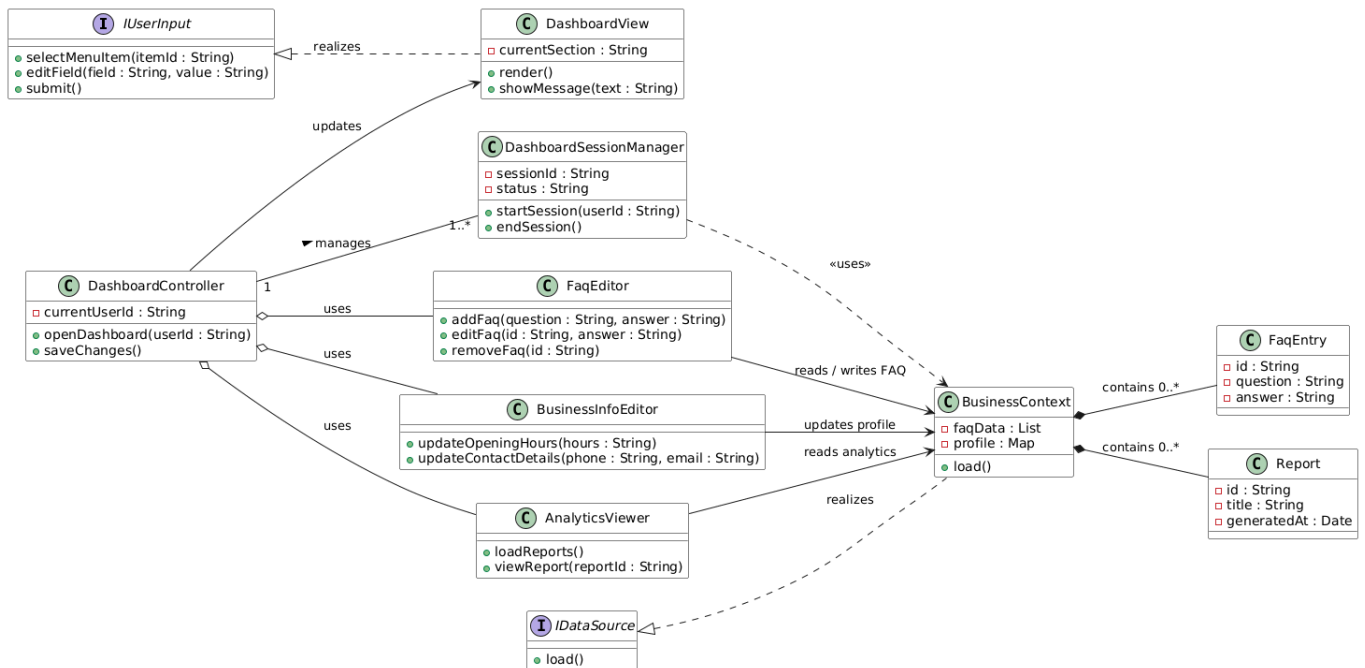


Figure 3.12 - Class Diagram – Business Web Portal (Dashboard)

4 System Implementation

This chapter presents the practical implementation of the proposed system. The application was developed as a full-stack web platform that allows a business owner to configure and monitor an AI voice assistant through a secure dashboard. The implementation combines a modern frontend, a server-side API layer, authentication and access control mechanisms, and a PostgreSQL database used for persistent storage.

The purpose of this chapter is twofold. First, it explains the technologies, programming languages, libraries, and implementation patterns used in the project. Second, it describes the application at code level, organized by functional modules, and shows representative code snapshots from the actual implementation.

4.1 Code-Level Description by Modules

The system was implemented using Next.js 16, React 19, and TypeScript. This combination was selected because it allows both the user interface and the backend API to be developed in the same project. As a result, the application has a simpler structure and deployment process than an architecture that separates frontend and backend into different repositories.

The frontend interface is built with React functional components and styled with Tailwind CSS 4. The backend logic is implemented through Next.js Route Handlers placed under the `app/api/` directory. Data persistence is handled using PostgreSQL through the `pg` library. User authentication relies on JWT tokens generated with `jose`, while passwords are hashed with `bcryptjs`.

From an implementation perspective, the project is structured into several clear modules:

- `app/` for pages and route-based UI;
- `app/api/` for backend endpoints;
- `app/dashboard/` for authenticated business management pages;
- `app/components/landing/` for the public presentation pages;
- `lib/` for shared logic such as authentication, database access, and schema initialization.

The following code snapshot illustrates the general project scripts and the main dependencies used in the implementation.

This technological stack supports a responsive web interface, a REST-style API, secure authentication, and persistent storage in a relational database.

4.1.1 Authentication, Access Control Module and Database

The authentication module was implemented to secure both the user interface and the API endpoints. The system uses password hashing and JSON Web Tokens. During registration, the password is never stored in plain text. Instead, it is transformed using `bcryptjs` with 12 salt rounds. After login or registration, the

```
1 {
2   "scripts": {
3     "dev": "next dev",
4     "build": "next build",
5     "start": "next start",
6     "lint": "eslint"
7   },
8   "dependencies": {
9     "bcryptjs": "^3.0.3",
10    "dotenv": "^17.3.1",
11    "jose": "^6.2.1",
12    "next": "16.1.6",
13    "pg": "^8.20.0",
14    "react": "19.2.3",
15    "react-dom": "19.2.3"
16  }
17 }
```

Listing 4.1 - package.json scripts and main dependencies used in the project.

server generates a signed JWT token that contains the user identifier and email address.

The token is stored in an HTTP-only cookie. This design was chosen because it reduces the risk of exposing the token to client-side JavaScript. The cookie is later checked in middleware and inside protected API endpoints.

The following code snapshot shows the core authentication helper functions.

At route level, the application uses middleware to protect dashboard pages, onboarding pages, and all API routes except the authentication routes. If a request does not contain a valid token, the user is redirected to the login page for normal pages, or the API returns a 401 Unauthorized response.

This module ensures that only authenticated users can access the management functionality of the application.

The persistence layer was implemented using PostgreSQL. A connection pool is created through the pg library, and all queries are executed using parameterized SQL statements. This reduces the risk of SQL injection and keeps the database layer predictable and secure.

The system schema is initialized programmatically using the `initDB()` function. This function creates the required tables if they do not already exist. The implemented schema contains seven main tables: `users`, `businesses`, `calls`, `assistant_settings`, `services`, `faqs`, and `working_hours`.

The following code snapshot shows the database connection and part of the schema initialization logic.

The `users` table stores authentication and account-level information. The `businesses` table stores the detailed company profile configured during onboarding. The `services`, `faqs`, and `working_hours` tables store the operational knowledge used by the voice assistant. The `calls` table stores call history data, while `assistant_settings` stores assistant behavior and activation state.

```

1 import bcrypt from "bcryptjs";
2 import { SignJWT, jwtVerify } from "jose";
3
4 const SALT_ROUNDS = 12;
5
6 export async function hashPassword(password: string): Promise<string> {
7   return bcrypt.hash(password, SALT_ROUNDS);
8 }
9
10 export async function verifyPassword(
11   password: string,
12   hash: string
13 ): Promise<boolean> {
14   return bcrypt.compare(password, hash);
15 }
16
17 export async function signToken(payload: { userId: string; email: string }) {
18   return new SignJWT(payload as Record<string, unknown>)
19     .setProtectedHeader({ alg: "HS256" })
20     .setIssuedAt()
21     .setExpirationTime("7d")
22     .sign(secret);
23 }

```

Listing 4.2 - Core authentication helper functions used for password hashing and JWT signing.

The registration and login module was implemented through dedicated API endpoints under `app/api/auth`. These endpoints validate user input, communicate with the database, generate tokens, and return structured JSON responses.

During registration, the application validates required fields, checks whether the email already exists, hashes the password, inserts the user into the database, and then creates the authentication cookie. During login, the application validates the supplied credentials and compares the password with the stored hash.

The following code snapshot shows the registration flow.

The login flow follows the same structure but replaces the insertion step with credential verification.

This module provides the entry point into the application and establishes the secure session used by the rest of the system.

4.1.2 Onboarding and Business Profile Module

After authentication, a newly registered user must complete an onboarding flow. This module was implemented as a multi-step form on the client side and a dedicated API endpoint on the server side.

The onboarding form collects the business description, address, services description, pricing notes, policies, communication style, and supported languages. The objective is to build a structured business profile that the assistant can later use as contextual knowledge.

The interface organizes the onboarding process into four steps: basic information, services and

```

1 export async function middleware(req: NextRequest) {
2   const token = req.cookies.get("token")?.value;
3
4   if (!token) {
5     if (pathname.startsWith("/api/")) {
6       return NextResponse.json({ error: "Unauthorized" }, { status: 401 });
7     }
8     return NextResponse.redirect(new URL("/login", req.url));
9   }
10
11  const payload = await verifyToken(token);
12  if (!payload) {
13    return NextResponse.redirect(new URL("/login", req.url));
14  }
15
16  return NextResponse.next();
17 }

```

Listing 4.3 - Middleware logic that blocks unauthenticated access to protected pages and APIs.

```

1 import { Pool } from "pg";
2
3 const pool = new Pool({
4   connectionString: process.env.DATABASE_URL,
5   ssl: { rejectUnauthorized: false },
6 });
7
8 export default pool;

```

Listing 4.4 - PostgreSQL connection pool configuration using the pg library.

pricing, policies and rules, and communication style with languages.

The following code snapshot shows the structure of the onboarding form state.

When the form is submitted, the backend either inserts a new business record or updates the existing one, creates default assistant settings if needed, and marks the user as onboarded.

This module is important because it transforms a newly created account into a configured business profile that can be managed through the dashboard.

4.1.3 Operational Management Modules: Dashboard, Monitoring, Services, FAQ, Working Hours, and Settings

The operational management layer allows the user to configure the information required by the assistant. This functionality is divided into several modules.

The services module implements CRUD operations for the company service catalog. Each service can include a name, description, price, and duration. The FAQ module allows the user to add common questions and answers that form a structured knowledge base. The working hours module stores weekly schedule information for each day of the week. The settings module stores assistant behavior such as

```

1 CREATE TABLE IF NOT EXISTS users (
2   id SERIAL PRIMARY KEY,
3   business_name TEXT NOT NULL,
4   owner_name TEXT NOT NULL,
5   email TEXT UNIQUE NOT NULL,
6   password TEXT NOT NULL,
7   phone TEXT NOT NULL,
8   business_type TEXT NOT NULL,
9   onboarded BOOLEAN NOT NULL DEFAULT false
10 );
11
12 CREATE TABLE IF NOT EXISTS services (
13   id SERIAL PRIMARY KEY,
14   user_id INTEGER REFERENCES users(id) ON DELETE CASCADE,
15   name TEXT NOT NULL,
16   description TEXT NOT NULL DEFAULT '',
17   price NUMERIC(10,2),
18   duration INTEGER
19 );

```

Listing 4.5 - Excerpt of the database schema initialization executed during startup.

greeting, fallback message, forwarding number, voice, language, and activation state.

The services creation endpoint is representative for the CRUD pattern used in the project.

The FAQ module follows the same approach, but for question-answer pairs.

The working hours module uses an upsert strategy so that all seven days can be updated efficiently in a single request.

The settings module also uses an upsert mechanism, but here the data is stored in a single row per user.

These modules form the core business configuration area of the application.

The dashboard module was implemented to provide the user with a centralized view of system activity. It shows total calls, missed calls, calls from the current day, assistant activation state, and recent call history.

At backend level, the dashboard endpoint is implemented as an aggregated data endpoint. Instead of sending multiple independent requests from the frontend, the server executes several database queries in parallel using `Promise.all()` and returns a single JSON response. This reduces response time and keeps the frontend simpler.

The following code snapshot shows this optimization pattern.

The call logs page extends this monitoring functionality through pagination. The algorithm is simple and efficient: the current page and page size are read from query parameters, then an offset is computed using the formula

$$offset = (page - 1) \times limit$$

```

1 export async function POST(req: NextRequest) {
2   await initDB();
3
4   const { businessName, ownerName, email, password, phone, businessType } =
5     await req.json();
6
7   const existing = await pool.query("SELECT id FROM users WHERE email = $1", [
8     email.toLowerCase().trim(),
9   ]);
10
11   if (existing.rows.length > 0) {
12     return NextResponse.json(
13       { error: "An account with this email already exists." },
14       { status: 409 }
15     );
16   }
17
18   const hashedPassword = await hashPassword(password);
19   const result = await pool.query(
20     `INSERT INTO users (business_name, owner_name, email, password, phone,
21     ↪ business_type)
22     VALUES ($1, $2, $3, $4, $5, $6)
23     RETURNING id, business_name, owner_name, email`,
24     [
25       businessName,
26       ownerName,
27       email.toLowerCase().trim(),
28       hashedPassword,
29       phone,
30       businessType,
31     ]
32   );

```

Listing 4.6 - Registration endpoint logic: validation, uniqueness check, password hashing, and user insertion.

This offset is then used in the SQL query to load only the required records.

This module gives the user a clear overview of the business assistant status and recent operational activity.

4.1.4 Frontend Interface Module

The frontend was implemented using reusable React components. The public part of the system includes a landing page with a navigation bar, hero section, features section, pricing section, demo section, and footer. The private part contains the dashboard pages for services, FAQs, working hours, calls, and settings.

The application uses client-side state management with React hooks such as `useState` and `useEffect`. The onboarding wizard, dashboard pages, and CRUD forms all rely on this pattern to fetch data, update state, and react to user actions.

```
1 const result = await pool.query("SELECT * FROM users WHERE email = $1", [
2   email.toLowerCase().trim(),
3 ]);
4
5 const valid = await verifyPassword(password, user.password);
6 if (!valid) {
7   return NextResponse.json({ error: "Invalid email or password." }, { status: 401 });
8 }
```

Listing 4.7 - Login endpoint excerpt: user lookup and password verification.

```
1 const [form, setForm] = useState({
2   description: "",
3   address: "",
4   servicesDescription: "",
5   pricingNotes: "",
6   policies: "",
7   communicationStyle: "professional",
8   languages: ["romanian"] as string[],
9 });
```

Listing 4.8 - Client-side onboarding form state structure.

The following code snapshot shows a representative pattern used in the dashboard to fetch protected data and redirect unauthenticated users.

At interface level, Tailwind CSS classes are used to create responsive layouts, loading states, form controls, and action buttons without maintaining a large custom CSS codebase.

Another important implementation aspect is the consistent handling of errors and the use of security-oriented coding practices. Most API handlers are wrapped in try/catch blocks and return predictable HTTP status codes such as 400, 401, 404, 409, and 500. This makes the application easier to debug and easier for the frontend to interpret.

The following code snapshot illustrates the general error-handling structure.

Security is also improved by using parameterized SQL queries, protected cookies, hashed passwords, token verification, and route-level access restrictions. Taken together, these measures make the implementation more robust and suitable for a real business management platform.

4.2 System Testing

System testing was approached at two levels: functional testing of the user flows and technical validation of the codebase. Because the project is a full-stack web application, the testing process focused mainly on validating end-to-end behavior across the interface, API routes, authentication logic, and persistence layer.

The main testing method used for this project is scenario-based functional testing. In this approach,

```

1 INSERT INTO assistant_settings (user_id) VALUES ($1)
2 ON CONFLICT (user_id) DO NOTHING;
3
4 UPDATE users
5 SET onboarded = true, updated_at = NOW()
6 WHERE id = $1;

```

Listing 4.9 - Onboarding persistence: create default assistant settings and mark the user as onboarded.

```

1 export async function POST(req: NextRequest) {
2   const userId = await getUserId(req);
3   const { name, description, price, duration } = await req.json();
4
5   const { rows } = await pool.query(
6     "INSERT INTO services (user_id, name, description, price, duration) VALUES ($1,
7     ↪ $2, $3, $4, $5) RETURNING *",
8     [userId, name.trim(), description?.trim() ?? "", price ?? null, duration ?? null]
9   );
10
11   return NextResponse.json({ service: rows[0] }, { status: 201 });

```

Listing 4.10 - Service creation endpoint following the CRUD pattern.

each major feature was executed as a user workflow and checked against the expected result. This method was appropriate because the application includes several interconnected modules, and their correctness depends not only on isolated logic but also on the interaction between frontend forms, cookies, API routes, middleware, and database operations.

The tested scenarios included:

- user registration with valid and invalid input;
- user login with correct and incorrect credentials;
- access restriction for protected routes;
- onboarding form completion and data persistence;
- adding, editing, and deleting services;
- adding, editing, and deleting FAQ entries;
- saving weekly working hours;
- updating assistant settings;
- reading dashboard statistics and paginated call logs.

Several tools and techniques were used during testing:

- the browser interface, for manual execution of the functional scenarios;
- the built-in Next.js runtime and API responses, for validating frontend-backend integration;
- PostgreSQL queries executed by the application, for verifying data persistence;

```
1 const { rows } = await pool.query(  
2   "INSERT INTO faqs (user_id, question, answer) VALUES ($1, $2, $3) RETURNING *",  
3   [userId, question.trim(), answer.trim()]  
4 );
```

Listing 4.11 - FAQ creation query using parameterized SQL.

```
1 INSERT INTO working_hours (user_id, day_of_week, open_time, close_time, is_closed)  
2 VALUES ($1, $2, $3, $4, $5)  
3 ON CONFLICT (user_id, day_of_week)  
4 DO UPDATE SET open_time = $3, close_time = $4, is_closed = $5;
```

Listing 4.12 - Working hours bulk update using an upsert pattern.

- ESLint, configured in the project scripts, for static analysis of the code;
- the Next.js production build command, intended to validate whether the application compiles correctly for deployment.

The following code snapshot shows the available project commands used for technical validation.

The functional testing confirmed that the main implemented modules behave consistently with the intended design. Authentication correctly blocks unauthorized access. The onboarding process stores business configuration data and marks the user as onboarded. The services and FAQ modules support creation, update, and deletion operations. The working hours and settings pages persist their values correctly. The dashboard endpoint aggregates data from several tables, and the call logs endpoint returns paginated results.

At implementation level, the testing also confirmed several technical aspects:

- password hashing is applied before storing user credentials;
- JWT cookies are created after successful registration and login;
- middleware restricts protected pages and protected API routes;
- SQL queries are scoped by `user_id`, preventing users from accessing other users' records;
- bulk updates for working hours are handled using `ON CONFLICT` upsert logic;
- dashboard statistics are retrieved efficiently through parallel queries.

The following code snapshot illustrates one of the tested access control conditions.

4.2.1 Problems Identified and Their Resolution

During implementation, the main category of problems involved synchronization between client-side forms and server-side validation. For example, features such as onboarding, settings, and CRUD pages required the frontend and backend to agree on field names, allowed values, and required properties. These issues were resolved by using TypeScript consistently, by validating required fields before database operations, and by returning explicit error messages from the API.

```

1 INSERT INTO assistant_settings (user_id, greeting, forward_number, fallback_message,
  ↳ voice, language, is_active)
2 VALUES ($1, $2, $3, $4, $5, $6, $7)
3 ON CONFLICT (user_id)
4 DO UPDATE SET
5   greeting = $2,
6   forward_number = $3,
7   fallback_message = $4,
8   voice = $5,
9   language = $6,
10  is_active = $7,
11  updated_at = NOW();

```

Listing 4.13 - Assistant settings upsert ensuring a single row per user.

```

1 const [totalCalls, missedCalls, todayCalls, assistantStatus, recentCalls, user] =
2   await Promise.all([
3     pool.query("SELECT COUNT(*)::int AS count FROM calls WHERE user_id = $1",
4       ↳ [userId]),
5     pool.query("SELECT COUNT(*)::int AS count FROM calls WHERE user_id = $1 AND
6       ↳ result = 'missed'", [userId]),
7     pool.query("SELECT COUNT(*)::int AS count FROM calls WHERE user_id = $1 AND
8       ↳ created_at >= CURRENT_DATE", [userId]),
9     pool.query("SELECT is_active FROM assistant_settings WHERE user_id = $1",
10      ↳ [userId]),
11     pool.query("SELECT id, caller, intent, duration, result, created_at FROM calls
12       ↳ WHERE user_id = $1 ORDER BY created_at DESC LIMIT 5", [userId]),
13     pool.query("SELECT business_name, owner_name, email, onboarded FROM users WHERE
14       ↳ id = $1", [userId]),
15   ]);

```

Listing 4.14 - Dashboard endpoint optimization: multiple queries executed in parallel.

Another important issue was ensuring secure access to protected resources. This was solved by combining two validation layers: middleware for route-level protection and token verification inside API endpoints. As a result, the system does not rely on frontend checks alone.

The persistence layer also required careful handling for records that should be created only once per user, such as assistant settings and working hours. This problem was solved using SQL upsert patterns with `ON CONFLICT`, which simplified updates and prevented duplicate records.

The following code snapshot shows the resolution strategy used for this case.

One practical limitation observed during validation is that technical commands such as `npm run lint` and `npm run build` must be executed from the project directory containing `package.json`. If they are started from the parent workspace folder, the scripts are not detected. This is an execution-context issue rather than an implementation defect, and it is solved by running the commands from the `teza` project folder.

The testing process shows that the implemented system fulfills the main functional requirements of the project. The application provides secure authentication, structured business onboarding, configurable

```
1 const page = Math.max(1, parseInt(url.searchParams.get("page") ?? "1"));
2 const limit = Math.min(100, Math.max(1, parseInt(url.searchParams.get("limit") ??
  ↳ "20")));
3 const offset = (page - 1) * limit;
```

Listing 4.15 - Pagination parameter parsing and offset calculation for call logs.

```
1 useEffect(() => {
2   fetch("/api/v1/dashboard")
3     .then((res) => {
4       if (res.status === 401) {
5         router.push("/login");
6         return null;
7       }
8       return res.json();
9     })
10    .then((d) => {
11      if (d) {
12        if (d.onboarded === false) {
13          router.push("/onboarding");
14          return;
15        }
16        setData(d);
17      }
18      setLoading(false);
19    })
20    .catch(() => setLoading(false));
21 }, [router]);
```

Listing 4.16 - Protected data fetch pattern with redirects for unauthenticated or not-onboarded users.

assistant knowledge and settings, and a dashboard for monitoring operational data. The combination of manual functional testing and technical validation methods was appropriate for the scope of the project and helped identify and solve integration and data consistency issues during development.

```

1 try {
2   const userId = await getUserId(req);
3   if (!userId) {
4     return NextResponse.json({ error: "Unauthorized" }, { status: 401 });
5   }
6
7   return NextResponse.json({ success: true });
8 } catch (error) {
9   console.error("Services POST error:", error);
10  return NextResponse.json({ error: "Internal server error." }, { status: 500 });
11 }

```

Listing 4.17 - Consistent API error handling using try/catch and HTTP status codes.

```

1 {
2   "scripts": {
3     "dev": "next dev",
4     "build": "next build",
5     "start": "next start",
6     "lint": "eslint"
7   }
8 }

```

Listing 4.18 - Project commands used for development and validation.

```

1 if (!userId) {
2   return NextResponse.json({ error: "Unauthorized" }, { status: 401 });
3 }

```

Listing 4.19 - Access control condition used across protected endpoints.

```

1 INSERT INTO assistant_settings (user_id, greeting, forward_number, fallback_message,
  ↳ voice, language, is_active)
2 VALUES ($1, $2, $3, $4, $5, $6, $7)
3 ON CONFLICT (user_id)
4 DO UPDATE SET
5   greeting = $2,
6   forward_number = $3,
7   fallback_message = $4,
8   voice = $5,
9   language = $6,
10  is_active = $7,
11  updated_at = NOW();

```

Listing 4.20 - Upsert strategy used to prevent duplicate assistant settings rows.

5 Product Documentation

This chapter documents the implemented VoiceAI web application from a practical perspective: how a first-time visitor discovers the product, how an administrator account is created, and how the initial setup is completed. The description is written as a user manual and covers both the public user experience (landing page and authentication) and the administrator experience (onboarding and, later, the dashboard). In the current version, the system does not include a dedicated customer portal; therefore, the administrator dashboard is the primary operational interface.

5.1 Landing Page

The landing page is the main public entry point of the application. It introduces the product, explains the main benefits, and provides clear navigation toward the authentication flow (/login and /register). The landing page is designed as a single scrollable page with a fixed header navigation bar and multiple content sections. A representative view is shown in Figure 5.1.

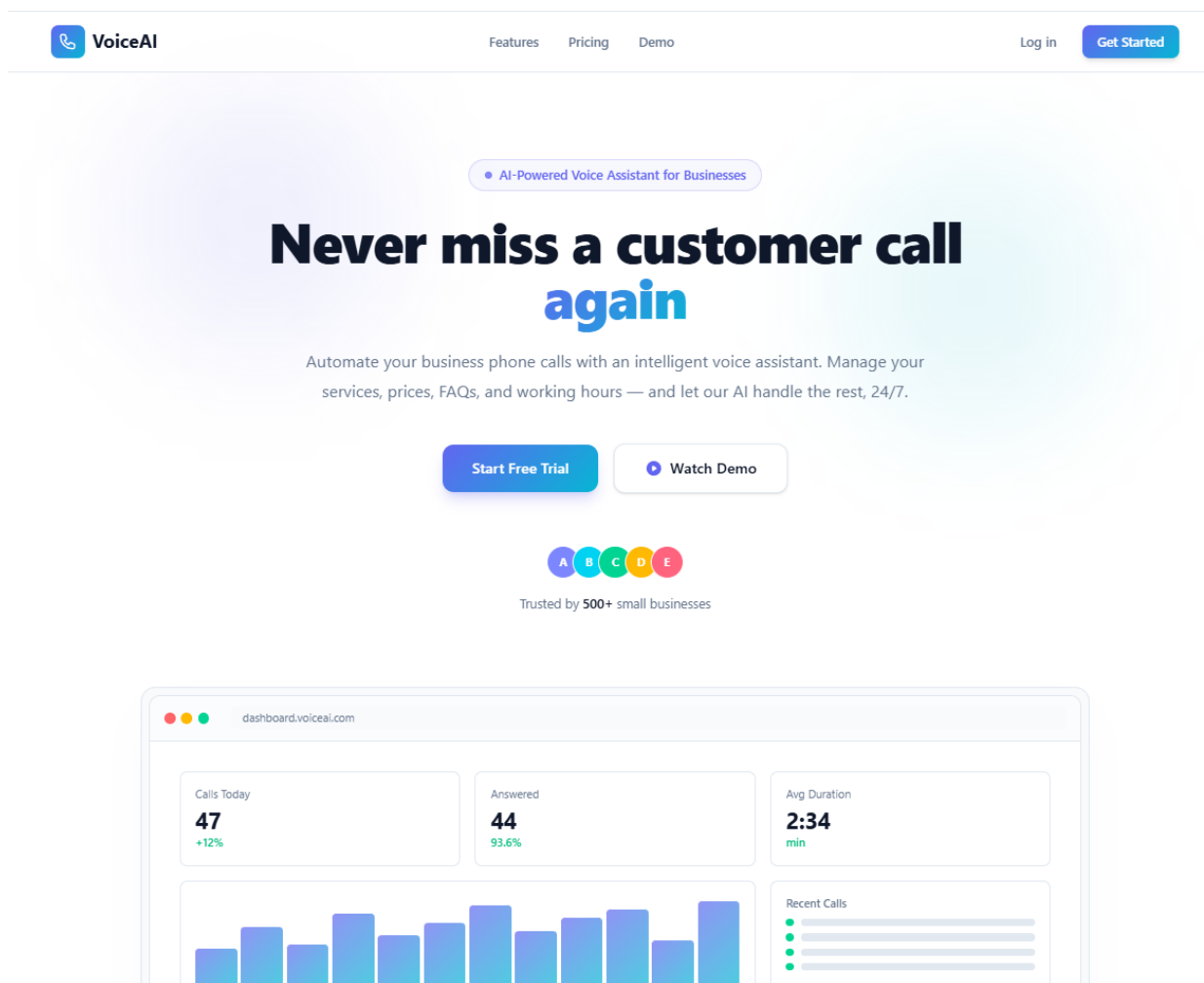


Figure 5.1 - Landing page interface.

At the top of the page, the navigation bar contains the VoiceAI logo on the left and several navigation links on the right. The logo labeled VoiceAI acts as a home action; selecting it keeps the user on the landing page and typically returns the scroll position to the top. The navigation links Features, Pricing, and Demo act as in-page anchors; selecting them scrolls the visitor to the corresponding section on the same page. The Log in control navigates to the login page for existing users. The Get Started button navigates to the registration page to start account creation.

The hero section contains a product category label (for example, AI-Powered Voice Assistant for Businesses), a main headline (“Never miss a customer call again”), and a short paragraph describing the value proposition: automating business phone calls with an assistant that uses configured services, prices, FAQs, and working hours. Two primary call-to-action buttons are visible. Start Free Trial redirects the visitor to the registration flow. Watch Demo scrolls the page to the demo section where a simulated call conversation is shown. Below these actions, the page includes a social proof indicator (for example, “Trusted by 500+ small businesses”).

A dashboard preview is displayed below the hero area to demonstrate the administrative interface. It typically shows example widgets such as calls today, answered calls, average duration, and a recent calls list. The preview is informational and does not provide direct interaction with the real dashboard.

The Features section explains the main capabilities of the system using feature cards, including company profile configuration, working hours scheduling, services and prices management, FAQ management, call analytics, and assistant settings. The Pricing section presents multiple subscription plans (for example Starter, Professional, Business), showing price, call limits, and included features, as well as call-to-action buttons such as Start Free Trial or Contact Sales. The Demo section simulates a phone conversation and reveals scripted messages step-by-step using a Continue Call control. Finally, the footer provides grouped navigation links under categories such as Product, Company, and Legal; depending on implementation scope, some links may be placeholders.

5.2 Registration Page

The first-time administrator flow consists of two consecutive phases. The first phase is account registration (/register), where the user creates an administrator account. The second phase is onboarding (/onboarding), where the newly created account is configured with business context and assistant preferences. Together, these steps complete the initial setup required before the dashboard can be used productively.

The registration page is the entry point for creating a new business administrator account. Its purpose is to collect the minimum identity and business metadata required to create a user record, establish an authenticated session, and redirect the new user to the onboarding wizard. The registration interface is

shown in Figure 5.2.

Create your account

Already have an account? [Log in](#)

Business Name

Bella's Hair Studio

Owner Name

John Smith

Email

john@example.com

Password

Minimum 6 characters

Phone Number

+1 (555) 000-0000

Business Type

Select a business type

Create Account

By registering, you agree to our [Terms of Service](#) and [Privacy Policy](#).

Figure 5.2 - Registration page interface.

The page is split into two columns. The left column contains the VoiceAI brand area and a short promotional message describing the product value. The right column contains the account creation form under the title “Create your account” and includes all interactive controls required for registration.

At the top of the form, a link labeled `Log in` is available for users who already have an account. Selecting this link navigates to the login page (`/login`) without submitting registration data.

The registration form contains the following fields in order: `Business Name`, `Owner Name`, `Email`, `Password`, `Phone Number`, and `Business Type`. `Password` is a protected input and displays a minimum length hint (“Minimum 6 characters”). `Business Type` is a dropdown selector used to categorize the business account.

The primary action button labeled `Create Account` submits the form to the backend registration endpoint. While the request is being processed, the interface may display a loading state and may temporarily disable the button to prevent duplicate submissions. Below the button, the page shows the agreement statement “By registering, you agree to our Terms of Service and Privacy Policy.”, where `Terms of Service` and `Privacy Policy` are displayed as links.

A typical registration procedure is as follows. The user opens `/register`, fills in the fields, then presses `Create Account`. On success, the system creates the account, establishes an authenticated session (commonly by setting an HTTP-only token cookie), and redirects the user to `/onboarding`. If the email already exists, the interface displays an error and the user should proceed to `Log in`. If validation fails, the

interface displays an error and the user must correct the input and resubmit.

After a successful registration, the application redirects the administrator to the onboarding wizard. The onboarding wizard completes the initial setup by collecting structured business information that the AI assistant will use later during customer calls. The wizard is titled `Set up your assistant` and is divided into four steps: `Basic Info`, `Services & Pricing`, `Policies & Rules`, and `Style & Languages`. The left side of the page includes a step indicator that displays progress using numbered circles and checkmarks.

Step 1 is shown in Figure ???. The heading is `Basic Information`. Two required fields are shown: `Business Description *` (a multi-line text area) and `Business Address *` (a single-line input). The `Continue` button advances to Step 2 after required fields are filled.

Step 2 is shown in Figure ???. The heading is `Services & Pricing`. The page contains two text areas: `What services do you offer?` and `General pricing details`. Navigation includes `Back` and `Continue`.

Step 3 is shown in Figure ???. The heading is `Policies & Rules`. The main control is a large text area labeled `Business policies`. Navigation includes `Back` and `Continue`.

Step 4 is shown in Figure ???. The heading is `Communication Style`. The `Tone of voice` group offers `Friendly`, `Professional`, and `Casual` as selectable options. The `Languages your assistant should speak` group offers `Română`, , and `English` as selectable language buttons, where at least one language should remain selected. The final submission is performed with `Activate Assistant`, which completes onboarding and redirects to `/dashboard`.

Completing onboarding is the final step of the first-time setup. Once onboarding is finished, the administrator can proceed to the dashboard pages to manage operational data such as working hours, services, FAQs, assistant settings, and call logs.

5.3 Login Page

The login page is used by an existing administrator to access the dashboard. The page provides a form for entering credentials and creating an authenticated session. A representative view is shown in Figure 5.3.

The form contains two input fields. The `Email` field is a text input where the user enters the account email. The `Password` field is a protected input that masks characters. To the right of the password label, a link labeled `Forgot password?` is visible; in the current implementation it is presented as a placeholder navigation element and may not lead to a complete password recovery flow.

The primary action button labeled `Sign In` submits the credentials to the backend login endpoint. While the request is processing, the interface may show a loading state and may temporarily disable the button to prevent repeated submissions. Below the button, the page displays an agreement statement indicating

that logging in implies acceptance of Terms of Service and Privacy Policy, which are presented as links.

The image shows a login page for VoiceAI. The left side has a blue gradient background. At the top left is the VoiceAI logo (a phone icon with a speech bubble). Below it, the text 'Welcome back' is displayed. Underneath is a paragraph: 'Log in to manage your voice assistant, review call logs, and keep your business running smoothly.' Below that is a testimonial card with a quote: 'Since using VoiceAI, we haven't missed a single customer call. It's like having a receptionist that never sleeps.' The card is attributed to Maria Johnson, Owner of Shine Dental Clinic. At the bottom left of the blue area is a small circular profile picture with the letter 'N'. The right side of the page is white. It has a heading 'Log in to your account'. Below the heading is a link: 'Don't have an account? Register'. There are two input fields: 'Email' with the value 'john@example.com' and 'Password' with masked characters. To the right of the password field is a link 'Forgot password?'. Below the input fields is a blue 'Sign In' button. At the bottom of the white area is a small text: 'By logging in, you agree to our Terms of Service and Privacy Policy.'

Figure 5.3 - Login page interface.

The page is split into two columns. The left column contains the VoiceAI logo, a welcome message (for example, “Welcome back”), a short paragraph explaining that login enables managing the assistant and viewing call logs, and a testimonial-style card. The right column contains the authentication form titled “Log in to your account”.

At the top of the form area, the page provides a navigation link labeled `Register`. Selecting this link redirects the visitor to the registration page (`/register`) without submitting any data.

If authentication is successful, the system sets the session cookie and redirects the user to the authenticated area. If the account has not completed onboarding, the user is expected to be redirected to `/onboarding`. If onboarding is completed, the user is redirected to the dashboard overview at `/dashboard`. If credentials are invalid, the page displays an error message and the user remains on the login form.

5.4 Dashboard Overview

The dashboard overview is the first page displayed after a successful login when onboarding is completed. Its purpose is to provide a quick summary of system activity and to offer navigation to the main configuration modules. A representative view is shown in Figure 5.4.

The dashboard layout is composed of a left sidebar and a main content area. The sidebar contains the VoiceAI logo at the top and a vertical navigation menu with the following items: Overview, Working Hours, Services, FAQ, Call Logs, and Settings. The currently selected page is visually highlighted (in

this case, Overview). At the bottom of the sidebar, a Log out control is displayed. Selecting Log out ends the session and returns the user to the login page.

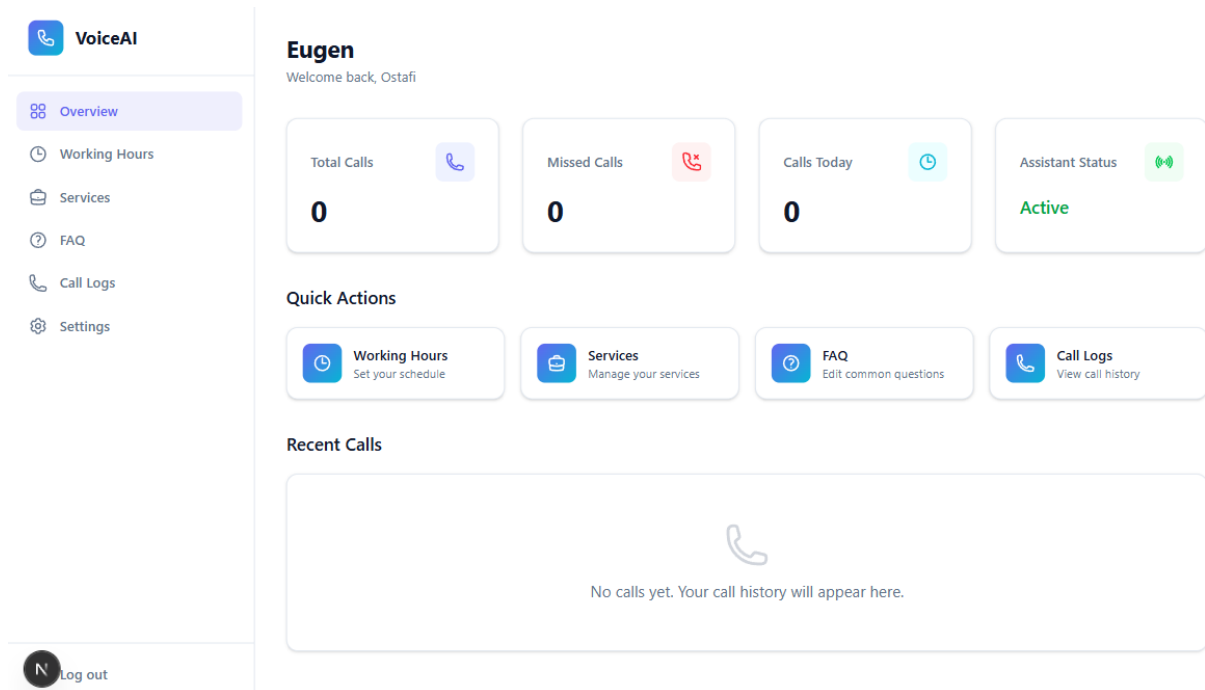


Figure 5.4 - Dashboard overview interface.

The main content area begins with a greeting that includes the account identity (for example a displayed name) and a short welcome subtitle. Below this header, the page presents a row of summary metric cards. The visible cards include Total Calls, Missed Calls, Calls Today, and Assistant Status. Each card contains a numeric value (or status text) and an icon. Assistant Status indicates whether the assistant is currently active.

Below the metrics, the Quick Actions section provides shortcut tiles that navigate directly to key pages. The visible shortcuts include Working Hours, Services, FAQ, and Call Logs. Selecting any tile redirects to the corresponding module page without needing to use the sidebar.

The lower part of the overview page contains a Recent Calls panel. When the account has no call history yet, the panel displays an empty-state message (for example: “No calls yet. Your call history will appear here.”). When calls exist, this area is expected to display a list or table of recent call records.

5.5 Working Hours

The working hours page is used to configure when the business is open for calls. The assistant can use this schedule to respond appropriately during and outside operating hours. A representative view is shown in Figure 5.5.

The page uses the same dashboard layout with the left sidebar navigation. In the main area, the

heading is **Working Hours** and a short subtitle explains that the user can set when the business is open for calls. The schedule editor is presented as a card containing one row per day of the week (Sunday through Saturday).

Day	Status	Opening Time	Closing Time
Sunday	☐ Closed	Closed all day	
Monday	☒ Open	09:00 AM	06:00 PM
Tuesday	☒ Open	09:00 AM	06:00 PM
Wednesday	☒ Open	09:00 AM	06:00 PM
Thursday	☒ Open	09:00 AM	06:00 PM
Friday	☒ Open	09:00 AM	06:00 PM
Saturday	☐ Closed	Closed all day	

Save Changes

Figure 5.5 - Working hours page interface.

Each day row includes the day label and an open/closed control. For days marked as closed, the row indicates that the business is closed for the full day (for example: “Closed all day”). For days marked as open, the row provides time inputs for opening and closing hours. The time inputs are displayed as two fields: an opening time and a closing time, separated by a small “to” label. Each time field includes a clock icon, indicating a time picker control.

The administrator edits the schedule by toggling the open/closed state for each day and adjusting opening and closing times where applicable. After editing, the changes are saved using the **Save Changes** button located below the schedule card. Pressing **Save Changes** sends the full weekly schedule to the backend. On success, the interface is expected to display a brief confirmation (for example a “Saved!” message) and preserve the updated values. If a network error occurs, the page is expected to show an error message and allow retry.

5.6 Services

The services page is used to manage the business service catalog, including adding new services, editing existing entries, and deleting services that are no longer offered. A representative view is shown in Figure 5.6.

The page contains a main heading **Services** and a short subtitle indicating that the user can manage the services the business offers. In the top-right area, a button labeled **+ Add Service** is visible. This button is used to open or focus the service creation form.

The screenshot displays the 'Services' management interface. On the left is a sidebar with navigation links: Overview, Working Hours, Services (highlighted), FAQ, Call Logs, and Settings. At the bottom of the sidebar is a 'Log out' button. The main content area has a heading 'Services' and a subtitle 'Manage the services your business offers.' in the top-left. In the top-right of this area is a '+ Add Service' button. Below the subtitle is a 'New Service' form card. The form contains a required 'Name *' field with a placeholder 'e.g. Haircut', a 'Description' field with a placeholder 'Brief description of this service', and two numeric input fields: 'Price (MDL)' with '0.00' and 'Duration (min)' with '30'. At the bottom of the form are 'Add Service' and 'Cancel' buttons. Below the form is a list of existing services. The first entry is 'haircut' with a short description 'haircut', a price of '350.00 MDL', and a duration of '60 min'. To the right of this entry are 'Edit' and 'Delete' buttons.

Figure 5.6 - Services page interface.

The service creation area is displayed as a form card titled **New Service**. The form contains a required field **Name ***, where the administrator enters the service title (for example: “Haircut”). A **Description** text area allows adding optional details about the service. Two additional numeric inputs are displayed in a row: **Price (MDL)** for the service price, and **Duration (min)** for the estimated duration in minutes.

At the bottom of the form, two action buttons are available: **Add Service** submits the new service to the backend and creates the record, while **Cancel** clears or closes the form depending on the current state. When the form is used in edit mode (after selecting an existing service to modify), the main submit button is typically expected to change to an update action (for example **Update**) while preserving the same field structure.

Below the form, the page displays the existing services as separate list items or cards. Each service entry shows the service name and may show additional details such as a short description, the price value, and the duration. Each entry includes two management controls: **Edit** and **Delete**. Selecting **Edit** loads the service data back into the form for modification. Selecting **Delete** initiates removal; the interface is expected to request confirmation before final deletion in order to prevent accidental data loss.

Together, these modules (**Working Hours** and **Services**) form part of the administrator config-

uration layer that supplies structured knowledge to the assistant and determines how the assistant should answer operational questions during customer calls.

5.7 FAQ / Knowledge Base

The FAQ page (also labeled FAQ / Knowledge Base) is used to maintain a structured set of question–answer pairs that the assistant can reuse when responding to callers. The goal of this module is to store consistent answers to common questions such as location, pricing rules, booking policies, or service details. A representative view is shown in Figure 5.7.

Figure 5.7 - FAQ / Knowledge Base page interface.

The page follows the standard dashboard layout with a sidebar navigation on the left and the module content on the right. In the sidebar, the FAQ item is highlighted, indicating the current page. The content area begins with the heading FAQ / Knowledge Base and a short explanatory subtitle indicating that the assistant will use these entries to respond to callers.

A button labeled + Add FAQ is visible in the top-right area of the page. This button is used to open or focus the FAQ creation form.

The main form card is titled New FAQ. It contains two required fields, each marked with an asterisk. Question * is a text input where the administrator writes the question as a caller would ask it (for example, “Where are you located?”). Answer * is a multi-line input where the administrator writes the official answer that the assistant should provide (for example, the exact address or directions). At the bottom of the form, the Add FAQ button submits the entry to the backend and creates a new FAQ record. The Cancel button

clears the input fields or exits the creation mode depending on the current state of the interface.

Below the form, existing FAQ entries are displayed as individual items. Each item shows the stored question and answer and includes two management controls: Edit and Delete. Selecting Edit loads the selected entry back into the form so it can be updated. Selecting Delete initiates removal; the interface is expected to request confirmation before permanent deletion to prevent accidental data loss.

5.8 Call Logs

The call logs page is used to view the history of calls handled by the assistant. This module serves as a monitoring interface and allows the administrator to review recorded interactions over time. A representative view is shown in Figure 5.8.

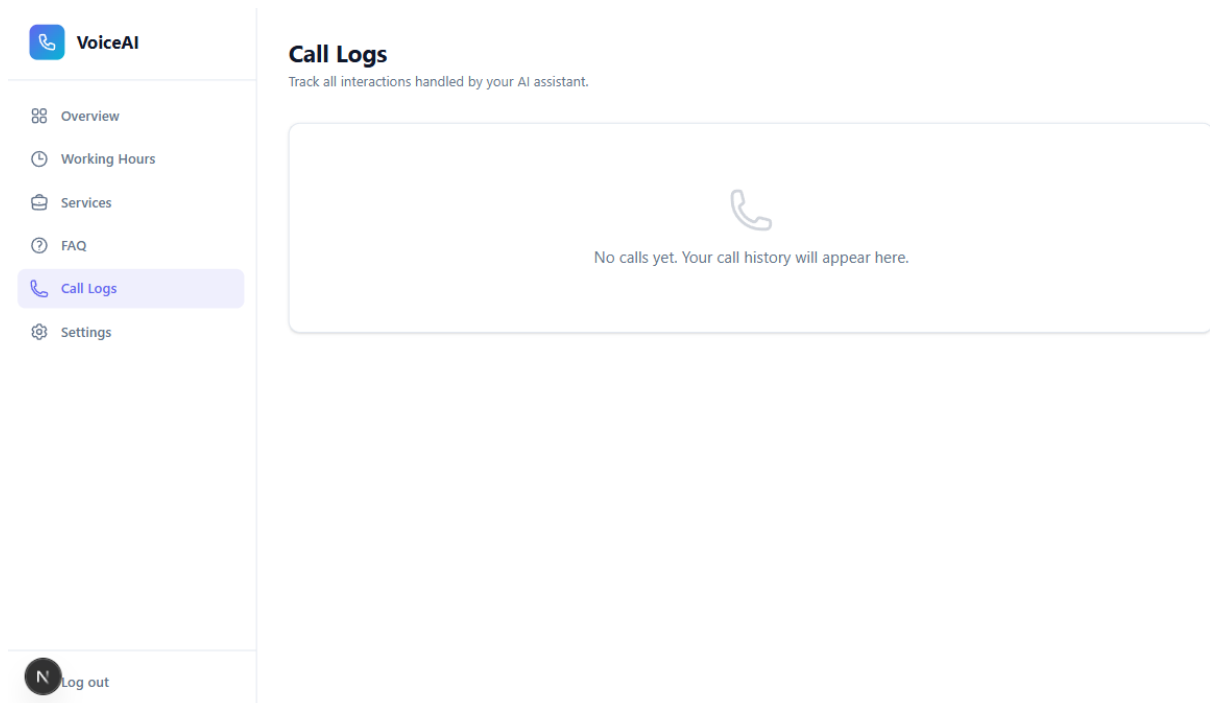


Figure 5.8 - Call Logs page interface.

The page uses the standard dashboard layout with sidebar navigation. The content area starts with the heading Call Logs and a short subtitle indicating that the page tracks all interactions handled by the assistant. The main panel is a large container intended to display call history entries.

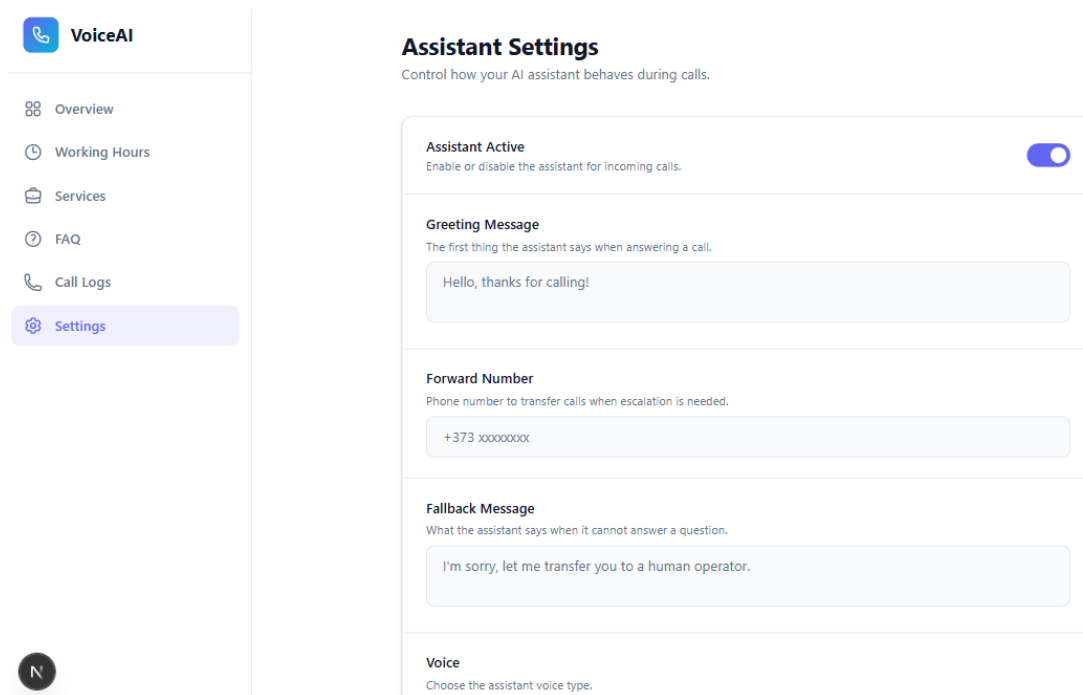
When no calls exist, the page shows an empty-state message (for example, “No calls yet. Your call history will appear here.”). When calls are available, this area is expected to display a table or list with call records. Each record typically includes key attributes such as timestamp, caller identifier, detected intent, duration, and call outcome. In implementations that support pagination, navigation controls such as Previous and Next appear at the bottom of the list to allow browsing through multiple pages of results.

The call logs module is important for operational monitoring because it provides evidence of assis-

tant activity and helps administrators validate whether the assistant is answering correctly, escalating when needed, and handling typical call volume.

5.9 Assistant Settings

The assistant settings page controls the operational behavior of the assistant during calls. It is used to enable or disable the assistant and configure the key messages and escalation settings that determine the caller experience. A representative view is shown in Figure 5.9.



The screenshot displays the 'Assistant Settings' page within a dashboard. On the left is a sidebar with a 'VoiceAI' header and a navigation menu containing 'Overview', 'Working Hours', 'Services', 'FAQ', 'Call Logs', and 'Settings' (which is highlighted). The main content area is titled 'Assistant Settings' with a subtitle 'Control how your AI assistant behaves during calls.' Below this, the settings are organized into five sections: 1. 'Assistant Active' with a toggle switch currently turned on. 2. 'Greeting Message' with a text input field containing 'Hello, thanks for calling!'. 3. 'Forward Number' with a text input field containing '+373 xxxxxxxx'. 4. 'Fallback Message' with a text input field containing 'I'm sorry, let me transfer you to a human operator.'. 5. 'Voice' with a link to 'Choose the assistant voice type.'

Figure 5.9 - Assistant Settings page interface.

The page follows the standard dashboard layout, with the Settings menu item highlighted in the sidebar. The content area begins with the heading `Assistant Settings` and a short subtitle indicating that the page controls assistant behavior during calls. The configuration is presented as a form divided into multiple labeled sections.

The first control is `Assistant Active`, implemented as a toggle switch. When enabled, the assistant is active for incoming calls. When disabled, the assistant is considered inactive; depending on the system design, calls may be handled differently (for example, redirected to a human or treated as unavailable). The toggle state is visually indicated by the switch position and color.

The `Greeting Message` field is a multi-line input that defines the first message the assistant says when answering a call. This message should be written in a natural and professional style consistent with the business. The `Forward Number` field is a single-line input where the administrator enters a phone number used for escalation or call transfer, meaning the assistant can forward a call to a human operator

when needed. The `Fallback Message` field is a multi-line input that defines what the assistant says when it cannot answer a question or needs to escalate; it typically apologizes and explains that the caller will be transferred.

The page also contains a `Voice` selection group with two options: `Female` and `Male`. Only one option is expected to be active at a time and the selected option is visually highlighted. The `Language` selection group defines the primary language used by the assistant. The visible options include `Romanian`, `Russian`, and `English`, and one option is expected to be selected as the primary language.

At the bottom of the settings form, a button labeled `Save Settings` submits the configuration to the backend. On a successful update, the interface is expected to display a confirmation message and preserve the updated values. If a validation error occurs (for example, an invalid phone number format depending on server-side rules) or if a network error occurs, the interface is expected to show an error message and allow the user to correct values and retry.

Together with the operational configuration pages (`Working Hours`, `Services`, and `FAQ`), the assistant settings page completes the administrator-level control surface required to configure a consistent and reliable caller experience.

CONCLUSIONS

This project addressed a real and increasingly important problem faced by small and medium enterprises (SMEs): the inability to provide consistent, immediate, and professional customer communication due to limited resources, staff, and time constraints. In a competitive environment where customer expectations are shaped by instant digital services, delayed responses and missed calls can directly impact customer satisfaction, business reputation, and revenue. The proposed Automated Voice Assistant System demonstrates how modern technologies such as speech recognition, natural language processing, and text-to-speech synthesis can be effectively combined to create a practical and accessible solution tailored specifically for SMEs. Unlike enterprise-level systems, which are often complex and expensive, this solution focuses on simplicity, affordability, and ease of deployment, making it suitable for businesses with limited technical expertise. Through the domain analysis, it was established that a large proportion of customer interactions in SMEs are repetitive and predictable. By automating these interactions, the system reduces the workload on employees, allowing them to focus on higher-value tasks that require human judgment and interaction. At the same time, the system improves response time, ensures consistency in communication, and enables 24/7 availability, thereby aligning SME capabilities with modern customer expectations. The system design and implementation demonstrate that it is technically feasible to integrate telephony services, AI-based voice processing, and a structured data layer into a cohesive architecture. The inclusion of a management dashboard ensures that business owners can easily update information such as working hours, services, and availability without requiring programming knowledge, which is essential for real-world adoption. However, the project also highlights several limitations and challenges. Voice-based interaction introduces complexity in terms of speech recognition accuracy, latency, and error handling, especially in noisy or unpredictable real-world environments. Additionally, considerations related to privacy, data protection, and system reliability must be carefully addressed in any production deployment. Future work can extend the system by improving conversational intelligence, supporting multiple languages, integrating appointment scheduling and order management, and optimizing performance for real-time interaction. Further research may also explore hybrid approaches that combine voice and messaging platforms to provide a unified customer communication experience. In conclusion, the Automated Voice Assistant System provides a viable and meaningful step toward bridging the gap between customer expectations and SME capabilities. By leveraging accessible AI technologies, the project contributes to improving operational efficiency, customer satisfaction, and competitiveness for small businesses, particularly in contexts such as the Republic of Moldova where SMEs form the backbone of the economy.

BIBLIOGRAPHY

1. EUROPEAN PARLIAMENT AND OF THE COUNCIL. *Regulation (EU) 2016/679 (General Data Protection Regulation)*. Published: Official Journal of the European Union. Apr. 2016.
2. EUROPEAN COMMISSION. *SMEs: Definition and Key Facts*. Published: Internal Market, Industry, Entrepreneurship and SMEs.
3. NATIONAL BUREAU OF STATISTICS OF THE REPUBLIC OF MOLDOVA. *Enterprises by Size Class / Small and Medium Enterprises Statistics*. Published: NBS Moldova.
4. GOVERNMENT OF THE REPUBLIC OF MOLDOVA. *Digital Transformation Strategy and Information Society Development Policy Documents*. Published: Official Government Resources.
5. PARLIAMENT OF THE REPUBLIC OF MOLDOVA. *Law on Personal Data Protection (Legea privind protecția datelor cu caracter personal)*. Published: Official Legislation Portal.
6. RADFORD, A. et al. *Robust Speech Recognition via Large-Scale Weak Supervision*. Published: arXiv preprint arXiv:2212.04356. 2022.
7. TWILIO INC. *Programmable Voice Documentation*. 2026.
8. POSTGRESQL GLOBAL DEVELOPMENT GROUP. *PostgreSQL Documentation*. 2026.
9. META. *WhatsApp – About*. 2026.
10. GOOGLE CLOUD. *Dialogflow Documentation*. 2026.
11. AMAZON WEB SERVICES. *Amazon Connect Documentation*. 2026.
12. TWILIO INC. *Twilio Flex Documentation*. 2026.
13. TWILIO INC. *Twilio Autopilot End-of-Life Notice*. 2026.